UNIVERSITY OF
CAMBRIDGE

Title

Elisabeta-Iulia (Julia) Dima

King's College

This dissertation is submitted on June, 2026 for the degree of Master of Philosophy

Abstract

My abstract ...

Declaration

This dissertation is the result of my own work and includes nothing which is the outcome of work done in collaboration except as declared in the Preface and specified in the text. It is not substantially the same as any that I have submitted, or am concurrently submitting, for a degree or diploma or other qualification at the University of Cambridge or any other University or similar institution except as declared in the Preface and specified in the text. I further state that no substantial part of my dissertation has already been submitted, or is being concurrently submitted, for any such degree, diploma or other qualification at the University of Cambridge or any other University or similar institution except as declared in the Preface and specified in the text. This dissertation does not exceed the prescribed limit of 60 000 words.

Elisabeta-Iulia (Julia) Dima
June, 2026

Acknowledgements

My acknowledgements ...

Contents

Chapter 1

Introduction

Understanding what neural networks *compute*, not only what they *output*, is the central problem of mechanistic interpretability. A model that correctly solves arithmetic problems or chains reasoning steps coherently need not be implementing anything resembling the algorithm a human would use. Whether it does, where in the network that computation lives, and whether it can be manipulated predictably are empirical questions that evaluation benchmarks cannot answer.

1.1 Mechanistic Interpretability

As neural networks scale from toy models to systems executed in consequential and complex settings, the gap between performance and understanding becomes untenable. Thus, a model achieving near-human accuracy on reasoning benchmarks cannot be straightforwardly understood. A system that has learned a spurious shortcut is indistinguishable from one that has learned the intended algorithm until it fails at the margin. The interpretability methods that accompanied earlier machine learning, such as saliency maps, attribution scores, and probing classifiers, operate in the input or output space and offer correlational accounts but are not mechanistic, as they describe what the model attends to, not what it computes.

Evidence for the tractability of mechanistic questions accumulated independently. Work on convolutional vision networks identified low-level features, including edge detectors, curve analysers, and frequency filters, that arise independently of architecture and training seed [**?**]. Analysis of transformer attention heads revealed heads implementing specific, describable operations, such as induction heads completing repeated token sequences and positional heads copying information from fixed offsets [**?**]. A direct demonstration of such mechanisms is the grokking phenomenon [**?**], which shows that small models trained on modular arithmetic initially memorise their training data and later undergo a phase transition, developing compressed algorithms that generalise across all held-out operand

pairs for the trained modulus. The transition is invisible to accuracy metrics evaluated on the training set. However, Fourier decomposition of the learned weight matrices reveals that the converged model implements modular arithmetic through specific trigonometric identities encoded in the residual stream, an algorithm with no surface similarity to the training objective [**?**].

Further work has confirmed that small transformers trained on arithmetic tasks converge on specific, inspectable algorithms, independent of surface statistical regularities in the data [**?** **?** **?** **?**].

The same interpretability considerations apply to large language models (LLMs), yet at a scale that makes manual circuit analysis infeasible. As LLMs are trained on vastly more diverse data and without explicit algorithmic curricula, they nonetheless achieve high accuracy on structured reasoning tasks. However, whether this reflects stable, localisable internal computation or brittle surface associations cannot be determined from evaluation alone. This distinction matters wherever model outputs yield real consequences.

Anthropic's *On the Biology of a Large Language Model* [**?**] proposes a reproducible methodology for approaching interpretability on a large language model (LLM). Combining sparse autoencoders (SAEs) with cross-layer transcoders (CLTs) and backward-pass attribution, it constructs dependency graphs that trace information flow from input tokens through intermediate features to output logits. The framework forms the methodological foundation of this thesis, which examines how structured reasoning is implemented within large instruction-tuned models and what the geometry of those representations reveals about the nature of computation.

1.2 Interpretability Methods

All four methods employed in this thesis operate on the residual stream of the transformer. Let $h_t^{(l)} \in \mathbb{R}^d$ denote the residual-stream vector at layer $l \in \{0, \ldots, L-1\}$ and token position t, with d the model dimension and L the total number of layers. Each layer contributes an additive update,

$$h_t^{(l+1)} = h_t^{(l)} + \mathrm{Attn}^{(l)}\left(H^{(l)}\right)_t + \mathrm{MLP}^{(l)}\left(h_t^{(l)}\right), \tag{1.1}$$

where $H^{(l)}$ denotes the full sequence of residual-stream vectors at layer l. This additive structure makes the residual stream a natural object for linear analysis, as each layer writes into a shared representation that accumulates across depth.

Cross-layer transcoders (CLTs) and sparse autoencoders (SAEs) decompose each MLP contribution into a sparse sum of interpretable directions [**?** **?**]. Given a pre-MLP hidden

state h, the transcoder encoder produces feature activations

$$a_f(h) = \mathrm{ReLU}\left(w_f^\top h + b_f\right), \quad f \in \{1, \ldots, F\}, \tag{1.2}$$

and the MLP output is approximated as $\sum_f a_f(h) \cdot d_f$, where $d_f \in \mathbb{R}^d$ are learned decoder directions. Sparsity ensures that at most a small active set $\mathcal{S}(h) \subset \{1, \ldots, F\}$ contributes for any given input, making each feature's role inspectable in isolation.

Attribution graphs assign a scalar influence score to each (feature, layer, position) triple with respect to a target output logit y_j [**?**]. Under the linearised backward pass, the attribution of feature f at layer l and position t is

$$\alpha_{f,l,t} = a_f\left(h_t^{(l)}\right) \cdot \left(d_f^\top \nabla_{h_t^{(l)}} y_j\right), \tag{1.3}$$

where $\nabla_{h_t^{(l)}} y_j$ is the gradient of the logit with respect to the residual stream. Composing attributions across layers yields a directed graph whose edges represent information flow from inputs to output.

Activation patching establishes causal claims [**?**]. Given a clean prompt x and a counterfactual prompt x', patching replaces the residual stream at layer l and position t,

$$\tilde{h}_t^{(l)} = h_t'^{(l)}, \tag{1.4}$$

and measures the resulting shift in output logit, $\Delta_j = y_j(\tilde{h}) - y_j(h)$. A non-zero Δ_j confirms that the patched representation is causally sufficient to alter the prediction. Constrained patching restricts the replacement to a targeted subset of positions or components, enabling finer localisation of causal dependence.

Steering vectors are extracted by contrasting two prompt sets, D_+ where a concept is present and D_- where it is absent, and computing their mean difference at a designated anchor position t^*,

$$\delta^{(l)} = \mathbb{E}_{x \in D_+}\left[h_{t^*(x)}^{(l)}\right] - \mathbb{E}_{x \in D_-}\left[h_{t^*(x)}^{(l)}\right]. \tag{1.5}$$

Under the linear representation hypothesis [**?**], $\delta^{(l)}$ approximates the axis along which the concept is encoded at depth l. Injecting $\alpha\,\delta^{(l)}$ into the residual stream at inference time provides a continuous lever on the concept representation without modifying model weights. The norm profile $\|\delta^{(l)}\|$ across layers is the primary signal used for concept localisation throughout this thesis. In combination, these four methods support mechanistic claims about how a model computes, not only whether it succeeds on a given input.

1.3 This Thesis

We take the open-access model `Qwen3-4B` as our subject. The primary contribution is a systematic concept localisation study across a diverse set of concepts spanning three structural categories: modular arithmetic (carry propagation in integer addition, GCD divisibility, and residue class membership), relational reasoning (transitive ordering and negation scope), and physical and causal plausibility (energy conservation and causal direction). The central hypothesis is that modular concepts, whose outcomes are determined by discrete thresholds or periodic relations, produce qualitatively different localisation signatures from relational and physical concepts, whose outcomes depend on monotone comparison or world knowledge.

The rest of this dissertation is structured as follows. Chapter 2 covers the relevant background. .....

Chapter 2

Related Work

2.1 Arithmetic algorithms in transformers

A productive line of mechanistic interpretability research has asked not merely whether transformers can perform arithmetic, but *how* they do so: what internal representations they form and what computational steps those representations support.

?] studied a one-layer transformer trained on modular addition and found that grokking, the phenomenon whereby generalisation appears suddenly after extended training on memorised data, corresponds to the emergence of a Fourier-based algorithm. The trained model encodes operands as superpositions of trigonometric features and composes them across layers to compute the result. The emergence of this structure can be tracked through interpretable progress measures even before the accuracy curve improves significantly.

?] revisited modular addition and showed that the same task admits two qualitatively distinct algorithmic solutions: a Fourier-based strategy they term the "clock" algorithm, and a lookup-table strategy they term the "pizza" algorithm. Which solution emerges depends on training configuration, demonstrating that identical supervision can produce structurally different circuits and that mechanistic diversity is not solely an artefact of random initialisation.

For standard integer addition, **?**] identified a carry-propagation circuit distributed across attention and MLP layers, bearing a structural resemblance to the long-addition algorithm familiar from elementary arithmetic. **?**] extended this account to subtraction, characterising how the same underlying circuit adapts to accommodate sign.

?] approached the problem from the perspective of training dynamics, tracing how the internal representations of operands and results cluster and align over the course of training on modular addition. Their analysis shows that organised algorithmic structure is not installed gradually but emerges through identifiable phase transitions, reinforcing the view that grokking marks a qualitative change in the model's internal organisation rather than a smooth accumulation of generalising evidence.

These results establish a strong prior that transformers trained specifically on arithmetic tasks do not merely interpolate over training examples but converge on discrete, inspectable computational procedures. All of the models studied, however, are small transformers trained from scratch on a single arithmetic task. The present work asks whether comparable structure is recoverable in `Qwen3`, a multi-billion-parameter instruction-tuned model trained on a broad natural-language corpus with no explicit arithmetic curriculum.

Chapter 3

Reproducibility

3.1 Reproducing the Addition Arithmetic Circuit

?] showed that Claude 3.5 Haiku, when presented with prompts of the form `calc:` a`+`b`=`, does not implement a symbolic carry algorithm but instead activates a family of sparse *lookup table* features whose firing concentrates on the cells of the operand space satisfying $a \bmod 10 = d_1$ and $b \bmod 10 = d_2$ for a specific digit pair (d_1, d_2). These features encode the ones digit of $d_1 + d_2$ and route this information directly to the output. The present reproduction asks whether the same mechanistic signature appears in Qwen3-4B, trained independently on a different corpus and with a different tokeniser, and whether the attribution methodology transfers faithfully to a model for which it was not originally calibrated.

3.1.1 Transcoders and the Linearised Computation Graph

Let $\mathcal{M}$ be a transformer with L layers, residual stream dimension d, and MLP sublayers $f_0, \ldots, f_{L-1}$. Mechanistic analysis replaces each f_ℓ with a pretrained *transcoder* $\mathcal{T}_\ell$, a sparse encoder–decoder whose weights are fit offline to reconstruct the MLP output while decomposing it into K interpretable directions. For a residual-stream vector $\mathbf{x} \in \mathbb{R}^d$ entering layer ℓ, the transcoder computes pre-activations

$$z_{\ell,k}(\mathbf{x}) = \left(\mathbf{W}_{\text{enc}}^{(\ell)}\,\mathbf{x} + \mathbf{b}_{\text{enc}}^{(\ell)}\right)_k, \tag{3.1}$$

so that feature k carries activation $a_{\ell,k}(\mathbf{x}) = \mathrm{ReLU}(z_{\ell,k}(\mathbf{x}))$, and the reconstructed MLP output is

$$\hat{f}_\ell(\mathbf{x}) = \mathbf{W}_{\text{dec}}^{(\ell)}\,\mathbf{a}_\ell(\mathbf{x}) + \mathbf{b}_{\text{dec}}^{(\ell)}, \tag{3.2}$$

where $\mathbf{W}_{\text{dec}}^{(\ell)} \in \mathbb{R}^{d \times K}$ and $\mathbf{a}_\ell(\mathbf{x})$ is the vector of all feature activations at layer ℓ.

Throughout this section a double subscript (ℓ, k) indexes features, with ℓ specifying the layer and k the index within that layer's dictionary, in place of the flat index f used in Chapter 1.3.

Attribution is computed on the *linearised* model, in which attention outputs are detached and treated as constants, so the residual stream propagates solely through the transcoder reconstruction path. This linearisation is the necessary condition for the attribution scores to be additive and interpretable as influence flows; it is also what constrains their faithfulness, a point examined in Section **??**.

Let T denote the position of the final input token, and let $y_j = \mathbf{v}_j^\top \mathbf{h}_T$ be the logit for output token j, where $\mathbf{h}_T$ is the pre-unembedding hidden state and $\mathbf{v}_j$ is the (demeaned) unembedding direction for token j. The *attribution score* of feature k at position t in layer ℓ is defined as

$$\alpha_{\ell,k,t} \;=\; a_{\ell,k,t} \cdot \frac{\partial y_j}{\partial a_{\ell,k,t}}, \tag{3.3}$$

the product of the feature activation $a_{\ell,k,t}$ with the gradient of the target logit with respect to that activation, consistent with Equation (1.3) of Chapter 1.3. Collecting all such scores over a given prompt yields a directed weighted graph, the *attribution graph*, in which nodes are (layer, position, feature) triples and edges carry the influence $\alpha_{\ell,k,t}$ from earlier features to later ones. The graph is then pruned by retaining only the nodes and edges that account for a fixed fraction of the total downstream influence on y_j, controlled by a node threshold τ_{node} and an edge threshold τ_{edge}.

3.1.2 Operand Plots and Lookup Feature Identification

For a prompt `calc: a+b=` , let T be the index of the `=` token. The *operand matrix* for feature (ℓ, k) is

$$M_{\ell,k}[a, b] \;=\; a_{\ell,k,T}(\texttt{calc: } a\texttt{+}b\texttt{= }), \qquad a, b \in \{0, \ldots, 99\}, \tag{3.4}$$

obtained by sweeping over the full 100×100 integer-pair grid. The geometry of $M_{\ell,k}$ encodes the feature's sensitivity in a directly interpretable form. A feature that tracks the raw sum $a + b$ produces diagonal stripes of constant activation; one that tracks a single operand produces horizontal or vertical bands; modular sensitivity to the ones digit of either operand produces a period-10 repeating structure. A feature that fires selectively for a specific pair of ones digits (d_1, d_2) produces a regular grid of isolated point clusters, concentrated on the 200 cells where $a \bmod 10 \in \{d_1, d_2\}$ and $b \bmod 10 \in \{d_1, d_2\}$. These *lookup table features* constitute 2% of the operand space; a uniform feature would achieve the same coverage by chance.

To identify lookup candidates automatically, each feature is assigned the concentration score

$$s_{\ell,k}(d_1, d_2) = \frac{\sum_{a,b} M_{\ell,k}[a,b] \mathbb{1}[a \bmod 10 \in \{d_1, d_2\},\ b \bmod 10 \in \{d_1, d_2\}]}{\sum_{a,b} M_{\ell,k}[a,b]}, \tag{3.5}$$

which measures the fraction of total activation mass that lies on the target grid. Features are filtered by a minimum peak-activation threshold to exclude dormant features, then ranked by $s_{\ell,k}$. The top-ranked feature for the pair $(d_1, d_2) = (6, 9)$ – the ones digits of the focus operands 36 and 59 – is taken as the reference lookup feature and subjected to causal validation. Two interventions are applied. In the *inhibition* experiment, the feature's activation is scaled by a factor $\beta \in \{1.0, 0.5, 0.0, -1.0, -5.0, -10.0\}$ before the decoder output is computed; if the feature is causally necessary, driving β below zero should monotonically weaken the logit for the token representing the correct ones digit of $6 + 9 = 15$. In the *substitution* experiment, the MLP inputs at all layers preceding the lookup feature's layer are replaced by those from the prompt `calc: 39+59=` , which has ones digits $(9, 9)$ rather than $(6, 9)$; if the identified feature mediates the computation, the output should shift from predicting ones digit 5 to ones digit 8.

Position-resolved feature sweep. **?**] observe that the features active during addition in Claude 3 Haiku differ systematically by token position: features at the operand tokens encode individual digit values, while features at the delimiter token encode their combination. To examine whether an analogous positional structure is present in Qwen3-4B, the operand sweep of Equation 3.4 was applied at three token positions in the prompt `calc:` *a*`+`*b*`=`: the ones digit of a (position 4), the ones digit of b (position 7), and the `=` token (position 8). At each position the top-15 CLT features by attribution edge weight were selected. Figure 3.1 shows the resulting matrices.

At position 4, the selected features activate in vertical bands with period 10 along the a axis. Activation is determined almost entirely by a mod 10 and is insensitive to b. At position 7 the pattern is transposed: horizontal bands with period 10 in b, independent of a. The symmetry between the two positions suggests that the ones digits of the two addends are encoded separately, with each operand token carrying information about that operand's own digit.

The `=` token displays a more varied set of activation patterns. Several features produce diagonal bands concentrated along contours of constant $a + b$, indicating sensitivity to the sum rather than to individual operand values. Others show a periodic point-cluster structure in which activation concentrates at cells where both a mod 10 and b mod 10 take

specific values; this is the joint-modular structure captured by the score in Equation 3.5, and it is the defining signature of the lookup table features described in Section **??**. A smaller number of features at this position show broader diagonal regions reflecting coarser magnitude sensitivity rather than exact digit values.

The positional separation visible in Figure 3.1 implies that ones-digit information is encoded before the two addends are jointly processed. At the operand token positions, no feature in the top-15 shows sensitivity to both a and b simultaneously; the cross-operand joint representation appears only at the `=` token. This ordering is consistent with the view that the network first reads off each digit independently and only subsequently computes a function of their combination.

3.1.3 Experimental Scope

The experiment is structured around four prompt collections, summarised in Table 3.1.

Table 3.1: Prompt collections used in the addition circuit reproduction. The operand grid drives the activation sweep; the focus prompt anchors the attribution graph.

Split	Template	N	Purpose
Operand grid	`calc:` a`+`b`=`	10,000	Feature activation sweep; $a, b \in \{0, \ldots, 99\}$
Focus prompt	`calc: 36+59=`	1	Primary attribution graph; answer `95`

The 10,000-prompt grid exhausts the integer-pair space $\{0, \ldots, 99\}^2$, covering all 100 distinct ones-digit combinations and providing sufficient support to identify modular activation patterns visually. The focus prompt `calc: 36+59=` follows Anthropic's exact choice, enabling direct comparison of attribution graphs across models.

3.1.4 Behavioural Verification

Before committing mechanistic resources to a single focus prompt, it is necessary to establish that the model solves the task reliably across the operand space. A circuit identified on a prompt the model cannot answer carries no mechanistic authority. The accuracy sweep therefore records, for every pair $a, b \in \{0, \ldots, 99\}$, the *teacher-forced confidence* at each output position. Let $\mathbf{y}^*(a, b) = (y_0^*, y_1^*, \ldots)$ denote the ground-truth answer tokens for $a + b$. The teacher-forced confidence at position k is

$$P_k(a, b) = P\Big(y_k = y_k^* \;\Big|\; y_0 = y_0^*,\ \ldots,\ y_{k-1} = y_{k-1}^*,\ \mathbf{x}(a, b)\Big)\,, \tag{3.6}$$

where $\mathbf{x}(a, b)$ is the token sequence for the prompt `calc:` a`+`b`=` and preceding positions are forced to their correct tokens. Equation (3.6) isolates the residual difficulty at each generation step by removing uncertainty due to error accumulation in prior positions.

Figure 3.2 plots $P_k(a, b)$ for $k \in \{0, 1, 2\}$. At $k = 0$ the model must predict the leading output token from the prompt alone; the grid mean is $\mu_0 = 0.233$, reflecting genuine uncertainty over a substantial fraction of the operand space. At $k = 1$, once the first token is teacher-forced to the correct value, mean confidence rises to $\mu_1 = 0.943$. By $k = 2$ it reaches $\mu_2 = 0.988$, with the only uncertain region being the upper-right triangle where $a + b \geq 100$ and the answer carries into a third digit. The cascade reveals that the central computational burden falls entirely on predicting y_0^*: subsequent tokens resolve almost automatically given the correct leading digit, a pattern that is consistent with a mechanism that maps directly from the operand ones-digits to the output rather than propagating carries sequentially.

Rather than varying smoothly with operand magnitude, the $k = 0$ panel of Figure 3.2 shows that confidence is organised into a period-10 grid aligned with the decimal ones-digit boundaries. Within each 10×10 block the within-block pattern repeats with the same geometry, and this periodicity implies that P_0 depends primarily on (a mod 10, b mod 10) rather than on the absolute values of a and b. This is the behavioural signature one would expect if the computation were implemented by features selective to specific ones-digit pairs.

Prediction entropy. Confidence $P_k(a, b)$ measures whether the top-ranked prediction is correct, but not how sharply the model has committed to any prediction. A cell with low confidence might correspond either to genuine distributional uncertainty or to a confident wrong prediction. The Shannon entropy of the truncated output distribution at position 0,

$$H(a, b) = -\sum_{j=1}^{K} p_{(j)}(a, b)\ \log_2 p_{(j)}(a, b), \tag{3.7}$$

where $p_{(1)}(a, b) \geq \cdots \geq p_{(K)}(a, b)$ are the top-K predicted probabilities, measures distributional spread independently of prediction correctness. Figure 3.3 plots $H(a, b)$ across the grid. Low-entropy cells (where the model commits decisively to a single token) form a period-10 grid aligned precisely with the high-confidence regions of the $k = 0$ panel of Figure 3.2. High-entropy cells (where the model distributes probability across several candidates) are concentrated at and near carry boundaries and for large operand values. The agreement between the two maps establishes that the model's decisiveness and its correctness are driven by the same underlying structure, rather than by one compensating for the other.

Carry complexity. Each pair (a, b) can be classified by the number of digit-column carries arising in the sum. Defining

$$\kappa(a,b) = \begin{cases} 0 & \text{no carry arises,} \\ 1 & \text{exactly one carry arises,} \\ 2 & \text{two or more carries arise,} \end{cases} \tag{3.8}$$

Figure 3.4 overlays the partition induced by κ on the confidence grid. If Qwen3-4B were implementing a symbolic carry algorithm, accuracy should deteriorate with carry complexity: pairs with $\kappa = 2$ should be hardest and pairs with $\kappa = 0$ easiest. The right panel of Figure 3.4 contradicts this prediction. Regions of identical carry class exhibit substantial internal variation in P_0, and the drops in confidence cut across carry boundaries rather than aligning with them. The period-10 grid structure of P_0 persists within each carry class, unmodulated by carry count. This dissociation between carry complexity and model accuracy constitutes behavioural evidence that the computation is not organised around carry propagation.

TODO: Conclusions...

3.2 Reproducing the Multilingual Circuit

?] showed that antonym completion in Claude 3.5 Haiku is not implemented by a monolithic circuit but by three functionally independent components, each localised in a distinct portion of the network. When presented with a prompt of the form *the opposite of "small" is "*, the model activates a family of transcoder features encoding the antonym *operation* (what semantic relation is being applied), the *operand* (the concept whose opposite is sought), and the output *language* (which language the response should appear in). By identifying source-exclusive and target-exclusive features for each component in isolation and injecting the latter while suppressing the former, they demonstrated that each component can be redirected independently without disrupting the others. The present experiment asks whether the same factorised structure is present in Qwen3-4B, a model trained on a different corpus with a different tokeniser.

The multilingual setting makes the factorisation hypothesis non-trivial to test. If the operation and operand components were language-specific, features identified from an English prompt would carry no predictive power over French or Chinese outputs. The hypothesis predicts instead that the features encoding the antonym relation and the concept of size are language-neutral directions in activation space, remaining active across prompts in all three languages, and that only the language component varies with the surface language of the input. This thesis reproduces three causal interventions on Qwen3-4B, targeting the operation, operand, and language components in turn, using the

transcoder-based causal methodology described below.

3.2.1 Exclusive Feature Identification

For each experiment, a *source* condition and a *target* condition are defined, differing along exactly one of the three components. Let $a_{\ell,k}^{\text{src}}$ and $a_{\ell,k}^{\text{tgt}}$ denote the activation of transcoder feature k at layer ℓ and token position p under the source and target conditions respectively. Taking the top-K active features in each condition, a feature is *source-exclusive* if it is strongly active in the source condition and weakly active in the target, and *target-exclusive* if the reverse holds. With exclusivity threshold $\rho \in (0, 1)$, the suppress set $\mathcal{S}^\ell$ and inject set $\mathcal{I}^\ell$ at layer ℓ are defined as

$$\mathcal{S}^\ell = \Big\{k \in \mathcal{F}_{\text{src}}^\ell : a_{\ell,k}^{\text{tgt}} < \rho \cdot a_{\ell,k}^{\text{src}}\Big\}, \qquad \mathcal{I}^\ell = \Big\{k \in \mathcal{F}_{\text{tgt}}^\ell : a_{\ell,k}^{\text{src}} < \rho \cdot a_{\ell,k}^{\text{tgt}}\Big\}, \tag{3.9}$$

where $\mathcal{F}_{\text{src}}^\ell$ and $\mathcal{F}_{\text{tgt}}^\ell$ are the top-K active feature indices at layer ℓ under the respective conditions. The threshold $\rho = 0.5$ was used throughout, so a feature qualifies as source-exclusive only if the target condition activates it at less than half the source-condition magnitude. The sets $\mathcal{S}^\ell$ and $\mathcal{I}^\ell$ are disjoint by construction: a feature active at comparable magnitude in both conditions satisfies neither criterion and belongs to neither set.

3.2.2 Causal Intervention Sweep

The intervention acts on the pre-MLP hidden state. Let $\mathbf{h}^{\ell,p} \in \mathbb{R}^d$ denote the post-RMSNorm input to the MLP at layer ℓ and token position p, the vector on which the transcoder encoder was trained to operate. For each layer ℓ in the designated range, $\mathbf{h}^{\ell,p}$ is projected through the transcoder encoder, the resulting feature representation is modified according to Equation (**??**), and the altered output supplants the standard MLP contribution at that position.

The feature activations produced by this projection are

$$\tilde{\mathbf{z}}^{\ell,p} = \text{ReLU}\Big(\mathbf{W}_{\text{enc}}^{(\ell)}\, \mathbf{h}^{\ell,p} + \mathbf{b}_{\text{enc}}^{(\ell)}\Big) \in \mathbb{R}^K. \tag{3.10}$$

Source-exclusive activations are set to zero and target-exclusive activations are augmented at intervention strength $\gamma \geq 0$,

$$\tilde{z}_k^{\ell,p} \;\leftarrow\; 0, \quad k \in \mathcal{S}^\ell; \qquad \tilde{z}_k^{\ell,p} \;\leftarrow\; \tilde{z}_k^{\ell,p} + \gamma\, v_k, \quad k \in \mathcal{I}^\ell, \tag{3.11}$$

where $v_k = a_{\ell,k}^{\text{tgt}}$ is the target-condition activation magnitude of feature k. The modified feature vector is then projected back to residual space via the transcoder decoder,

$$\tilde{\mathbf{o}}^{\ell,p} = \mathbf{W}_{\text{dec}}^{(\ell)}\, \tilde{\mathbf{z}}^{\ell,p} + \mathbf{b}_{\text{dec}}^{(\ell)}, \tag{3.12}$$

and computation propagates with $\tilde{\mathbf{o}}^{\ell,p}$ in place of the standard MLP output at position p and layer ℓ, all other positions and layers remaining undisturbed. The next-token probability for each monitored vocabulary index t at the final position T is

$$\pi_t(\gamma) = \Big[\mathrm{softmax}(\mathrm{logits}_T)\Big]_t. \tag{3.13}$$

The sweep is conducted over $\gamma \in \{0,\, \delta,\, 2\delta,\, \ldots,\, \gamma_{\max}\}$, yielding one probability curve π_t per monitored token. If the targeted component is causally sufficient for the corresponding aspect of the output, π_t for the target-condition token should rise monotonically with γ while the source-condition curve declines, constituting a directional redistribution of probability mass.

3.2.3 Three Causal Interventions

The three interventions differ in which component they target, at which token position features are identified, and over which layer range the modification is applied. The layer ranges are grounded in the decomposition reported by **?**] for Claude 3.5 Haiku: the operation component occupies the middle third of the network, the operand the early third, and the language component the first fifth. For Qwen3-4B with $L = 36$, these correspond to

$$\mathcal{L}_{\mathrm{op}} = \Big\{\big\lfloor \tfrac{L}{3} \big\rfloor,\, \ldots,\, \big\lfloor \tfrac{2L}{3} \big\rfloor - 1\Big\} = \{12, \ldots, 23\}, \tag{3.14}$$

$$\mathcal{L}_{\mathrm{oper}} = \Big\{0,\, \ldots,\, \big\lfloor \tfrac{L}{3} \big\rfloor - 1\Big\} = \{0, \ldots, 11\}, \tag{3.15}$$

$$\mathcal{L}_{\mathrm{lang}} = \Big\{0,\, \ldots,\, \max\Big(1, \big\lfloor \tfrac{L}{5} \big\rfloor\Big) - 1\Big\} = \{0, \ldots, 6\}. \tag{3.16}$$

These ranges are adopted from Anthropic's analysis of a distinct architecture and are not derived from the geometry of Qwen3-4B itself. Whether they correctly delimit the relevant layer segments for this model is an empirical question that the results address.

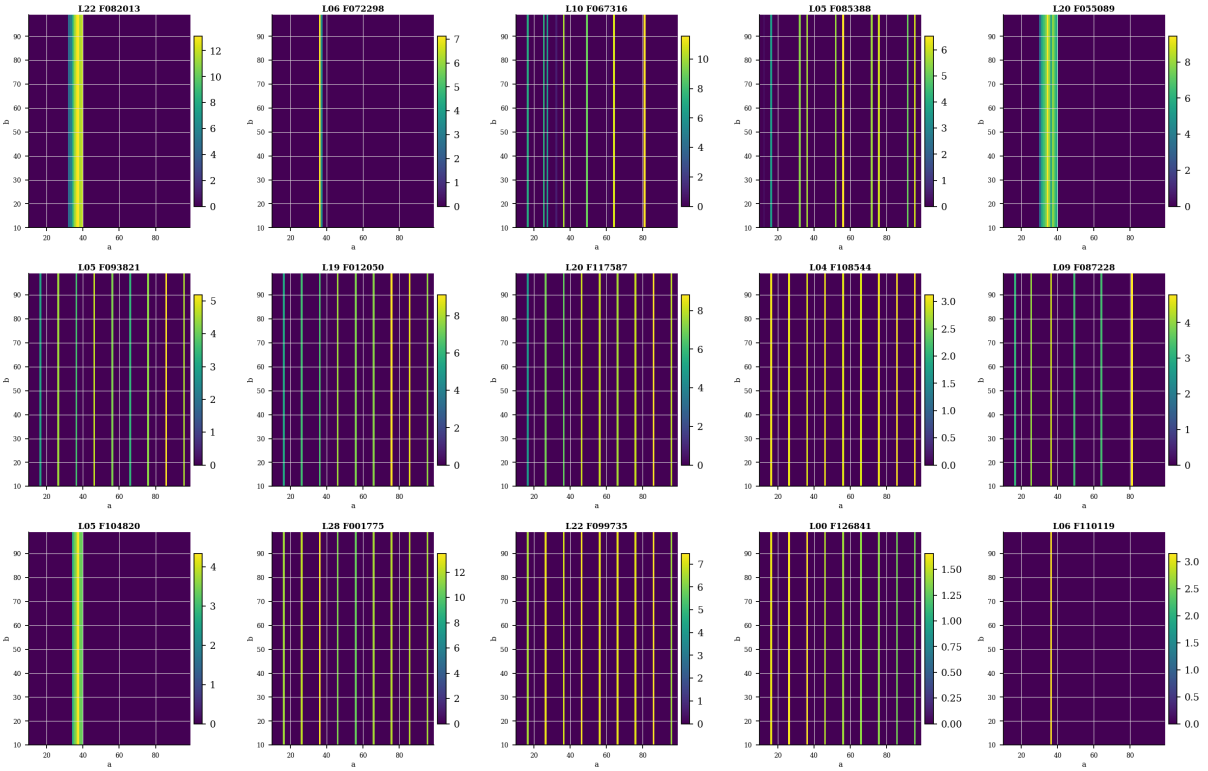
Operation swap. The antonym task is presented in English as *the opposite of "small" is "*, with the alternative prompt *a synonym of "small" is "* as the target condition. Features are identified at the final token position $p = T$ over layers $\mathcal{L}_{\mathrm{op}}$. The resulting exclusive partition $(\mathcal{S}^\ell, \mathcal{I}^\ell)$ is transferred without modification to antonym prompts in English, French, and Chinese. The monitored tokens are the antonym surface forms in each language and their synonym counterparts. If the operation features are language-neutral, the English-derived partition should redirect the output toward synonyms across all three languages.

Operand swap. The source condition is the English antonym prompt for "small" and the target is the English antonym prompt for "hot", *the opposite of "hot" is "*. Features are identified not at the final position but at the *operand divergence position* p^*, the first index at which the token sequences of the two English prompts diverge. This is the position at which the words "small" and "hot" enter the context respectively, and it is where the operand concept is hypothesised to be encoded. The exclusive partition is computed over $\mathcal{L}_{\text{oper}}$ and applied at the analogous divergence position in each language-specific antonym prompt. The monitored tokens are the antonym surface forms and the surface forms denoting cold in each language.

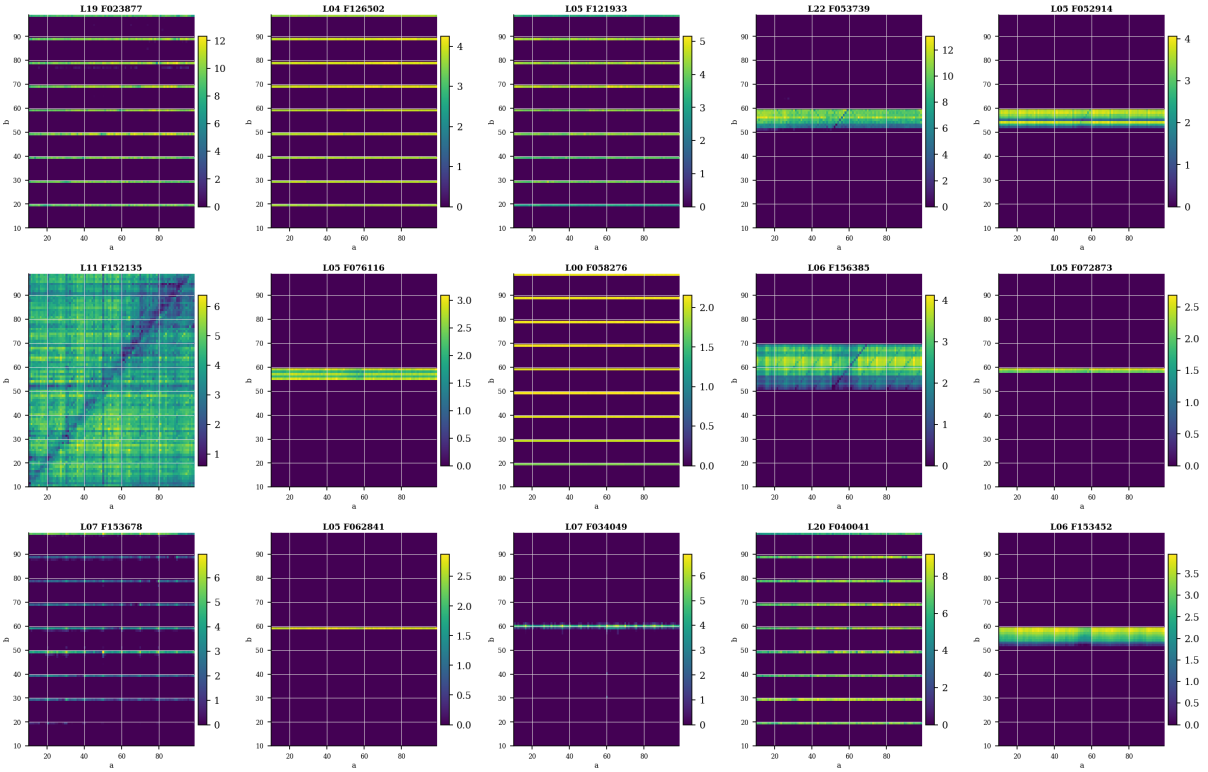
Language swap. Three directed language pairs are examined: English to Chinese, Chinese to French, and French to English. For each pair, both the source and target activations are drawn from antonym prompts in the respective languages, and the exclusive partition encodes the contrast between two language contexts rather than two semantic conditions. The partition is computed at the final position over $\mathcal{L}_{\text{lang}}$ and the intervention acts on the source-language prompt. The monitored tokens are the antonym surface forms in both languages. If the language component is independently localised, substituting target-language features for source-language ones should redirect the output toward the target-language antonym, leaving the underlying semantic relation undisturbed.

3.2.4 Results

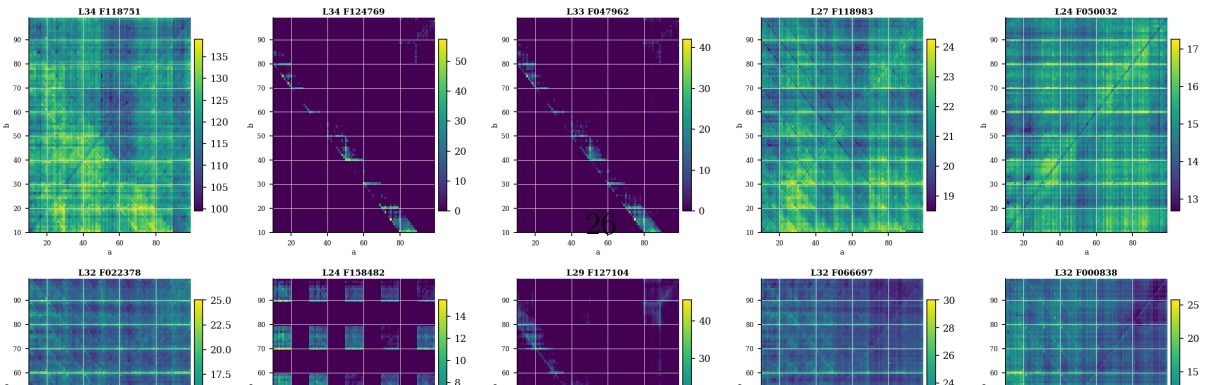
Prompt: "calc: 36+59=" token position: ones(a)



Prompt: "calc: 36+59=" token position: ones(b)



Prompt: "calc: 36+59=" token position: =



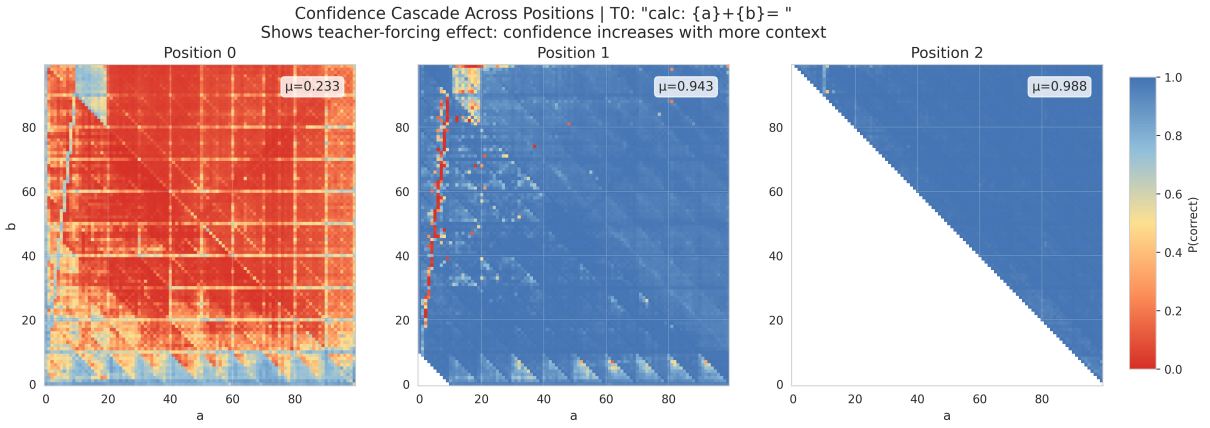


Figure 3.2: Teacher-forced confidence $P_k(a, b)$ (Eq. 3.6) across the 100×100 operand grid for positions $k = 0, 1, 2$. Each cell encodes the probability that Qwen3-4B assigns to the correct k-th output token when all preceding tokens are forced to their ground-truth values. Grid means are $\mu_0 = 0.233$, $\mu_1 = 0.943$, and $\mu_2 = 0.988$. The leftmost panel reveals a period-10 grid structure aligned with the decimal ones-digit boundaries: columns and rows separated by ten units carry systematically different baseline confidence levels, implying that P_0 depends primarily on (a mod 10, b mod 10) rather than on operand magnitudes. Empty cells in the middle panel correspond to pairs with $a + b < 10$, whose single-digit sum has no second output token; empty cells in the rightmost panel correspond to pairs with $a + b < 100$, whose two-digit sum has no third digit.

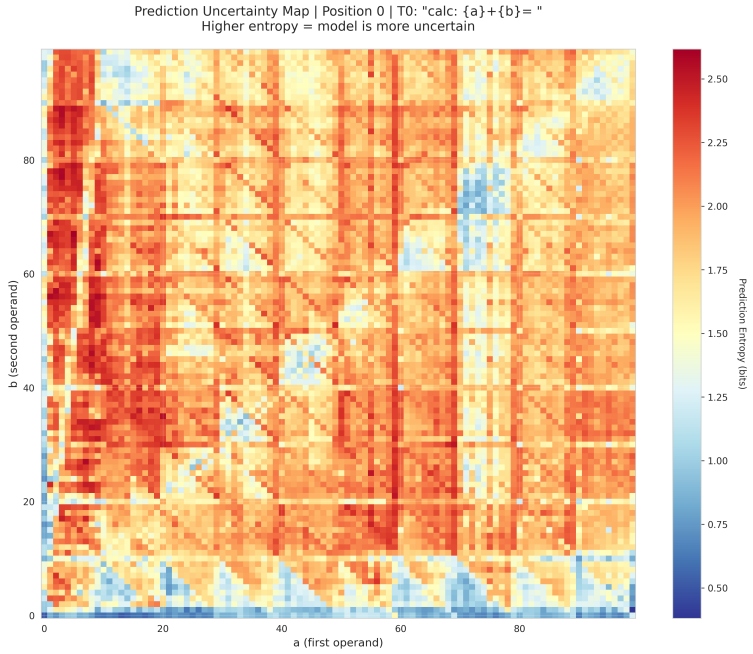


Figure 3.3: Prediction entropy $H(a, b)$ (Eq. 3.7) at position $k = 0$. Low-entropy cells (blue) form a period-10 grid aligned with the high-confidence cells visible in the $k = 0$ panel of Figure 3.2, indicating that the model is both decisive and correct on precisely the pairs where lookup features activate strongly. High-entropy cells (red) concentrate near carry boundaries and for large operand values, reflecting genuine distributional indecision rather than merely confident errors.

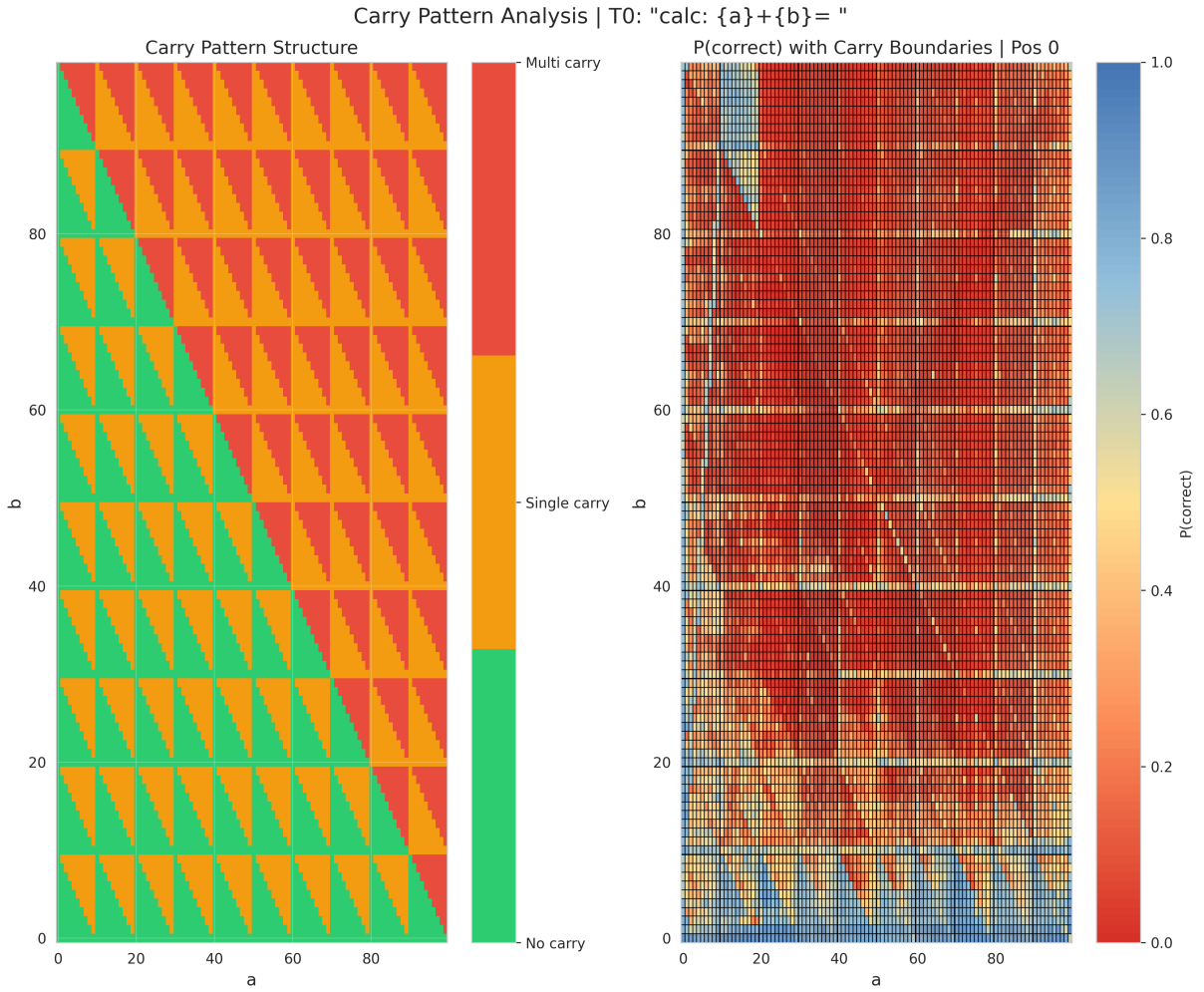


Figure 3.4: Left: carry-class partition $\kappa(a, b)$ (Eq. 3.8) of the operand grid, coloured by carry count. Right: first-token confidence $P_0(a, b)$ with carry-boundary contours superimposed. The period-10 structure of P_0 persists within each carry class, and drops in confidence do not align with the no-carry/single-carry/multi-carry boundaries, ruling out a symbolic carry algorithm as the underlying mechanism.

Chapter 4

Concept Localisation and Intervention

Mechanistic analysis of `Qwen3-4B` on integer arithmetic, following the methodology of **?**], reveals that the residual stream at the position corresponding to the completed arithmetic expression contains information relevant to downstream answer generation. Prior work has shown that simple linear probes trained on hidden representations can accurately decode both arithmetic outputs and model correctness from internal activations, indicating that these representations are often linearly accessible [**?**]. This observation motivates a concrete intervention. The mean difference in hidden state between prompts on which the model succeeds and those on which it fails defines a direction in representation space that, when added at inference time, should reinforce correct computation. To act on this hypothesis precisely, we require a mechanism that identifies *where* within a prompt a given primitive appears and with what confidence, avoiding a uniform perturbation across all token positions. We therefore design a *PrimitiveRouter*, a set of four learnable soft finite-state machines trained jointly on synthetic arithmetic expressions to detect which primitive operation is active at each token position. The router converges to perfect accuracy on its held-out validation set within 30 epochs, providing a reliable positional signal. Its output gates the injection of contrastive steering vectors extracted from the model's own activations, producing a closed-loop intervention that requires no modification of model weights and that handles prompts containing multiple primitives simultaneously. Figure 4.1 gives an overview of the complete pipeline.

4.1 Token Predicate Abstraction

Arithmetic expressions share structural patterns that are independent of the specific operand values they contain. The numeral thirty-seven and the numeral one thousand play structurally identical roles in their respective expressions, and what varies is only

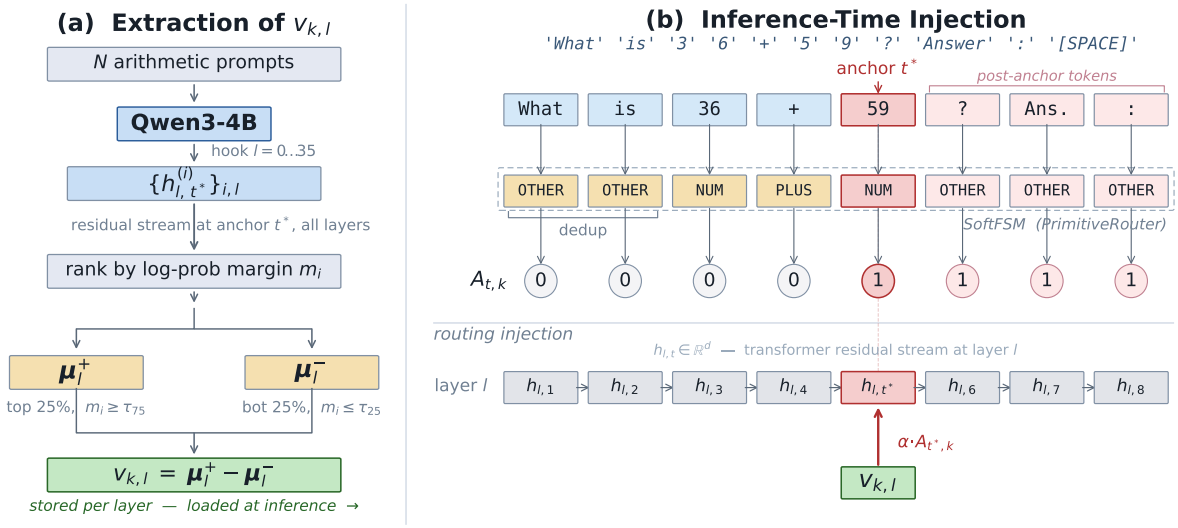


Figure 4.1: Overview of FSM routing and local steering on the prompt `What is 36+59? Answer:`. Tokens are mapped rule-based to predicates; consecutive identical predicates are collapsed (dedup) so that multi-digit numerals resolve to a single `NUM` step regardless of how many tokens the tokeniser produces. The SoftFSM processes the predicate sequence and outputs per-position activation values $A_{t,k}$ for the target primitive k. The position where the complete pattern first fires (token `9`, the second operand) is identified as the anchor t^*; the prompt continues with three further tokens (`?`, `Answer`, `:`) after the anchor, illustrating that the anchor need not coincide with the final token of the prompt. At inference time the steering vector $v_{k,l}$ is injected into the residual stream of `Qwen3-4B` at layer l and position t^*, scaled by $\alpha \cdot A_{t^*,k}$. The final space token is added to encourage the model to generate the answer token, instead of closing punctuation marks.

the surface form. We exploit this by defining a shared predicate vocabulary of eleven categories (Table 4.1), each mapping a class of surface tokens to a structural role. The mapping is deterministic and rule-based, requiring no learned components. Multi-digit integers, regardless of their magnitude, all map to `NUMBER`; operator symbols and their written equivalents map to the corresponding operator predicate; unrecognised tokens map to `OTHER`.

`Qwen3` tokenises multi-digit integers digit-by-digit, so the string `37` produces two separate tokens. To ensure that the predicate sequence seen by the router is independent of this tokenisation granularity, consecutive tokens that map to the same predicate are collapsed into a single group. The token sequence `["3", "7", "+", "1", "1", "5"]` therefore produces the predicate sequence `[NUMBER, PLUS, NUMBER]`, which is structurally identical to the sequence produced by a word-level tokeniser on the same expression.

Table 4.1: Shared predicate vocabulary. Each predicate covers all surface forms listed; the mapping is value-independent.

Predicate	Surface forms
NUMBER	Any integer or decimal literal
PLUS	+, plus, add
MINUS	-, minus, subtract
TIMES	*, ×, ·, times, mul, multiply
DIV	/, ÷, div, divide
MOD	%, mod, modulo
EQUALS	=, ==, equals, is
OPEN_PAREN	(, [, {
CLOSE_PAREN	),], }
COMMA	,
OTHER	Any unrecognised token

4.2 The Soft FSM Router

4.2.1 Architecture

We model each arithmetic primitive as a soft, learnable finite-state machine (SoftFSM) operating on the predicate sequence. Rather than maintaining a one-hot state indicator, the machine tracks a probability distribution over S latent states, which allows gradients to flow through transitions and makes training by standard gradient descent possible. At each predicate step t, the state is updated by

$$s_{t+1} = \text{softmax}(s_t \, T_{p_t}) \,, \tag{4.1}$$

where $s_t \in \mathbb{R}^S$ is the current state vector (with $s_0 = e_1$, all probability mass in state 0), and $T_{p_t} \in \mathbb{R}^{S \times S}$ is the learned transition matrix for predicate p_t. A scalar readout at each step produces the activation value for primitive k,

$$A_{t,k} = \sigma\Big(w_k^\top s_t + b_k\Big) \,, \tag{4.2}$$

where $w_k \in \mathbb{R}^S$ and $b_k \in \mathbb{R}$ are learned readout parameters, and σ denotes the sigmoid function. The value $A_{t,k} \in [0, 1]$ therefore measures how confidently the router believes that primitive k is active at predicate step t.

A *PrimitiveRouter* module wraps $K = 4$ independent SoftFSM instances (addition, subtraction, multiplication, modular reduction) sharing the same predicate vocabulary but with separate transition matrices and readout parameters. The router produces $A \in \mathbb{R}^{B \times T \times K}$. Each SoftFSM uses $S = 4$ latent states, contributing $P \cdot S^2 + S + 1 = 11 \times 16 + 5 = 181$ parameters per primitive (176 for the transition matrices, 4 for the readout weights, and 1 for the readout bias), giving a total of $K \times 181 = 724$ parameters

across all four primitives, a negligible overhead relative to the main model.

Transition matrices are initialised with a strong identity bias, $T_p \mathrel{+}= 3I$, so that each machine begins as a "stay put" automaton and learns to deviate only on structurally relevant predicates. Readout biases are initialised to $b_k = -2$, keeping activations near zero at initialisation so that the router does not fire spuriously on arbitrary inputs.

4.2.2 Supervised Training

The router is trained on synthetically generated arithmetic expressions with rule-based per-token labels. For each example, the label $y_{t,k} \in \{0, 1\}$ is set to 1 at all positions after the complete pattern for primitive k has appeared in the expression, and to 0 before. This cumulative, once-set labelling reflects the router's role as a detector of expression completion rather than a local classifier. For the expression `37 + 115`, the addition label becomes 1 at the second `NUMBER` token and remains 1 for all subsequent tokens, while all other primitive labels remain 0.

Training minimises binary cross-entropy averaged over all valid token positions and all primitives,

$$\mathcal{L} = -\frac{1}{|\mathcal{V}|} \sum_{(b,t) \in \mathcal{V}} \sum_{k=1}^{K} \Big[y_{t,k} \log A_{t,k} + (1 - y_{t,k}) \log(1 - A_{t,k}) \Big], \tag{4.3}$$

where $\mathcal{V}$ is the set of valid (batch, position) index pairs, excluding padding. The learning rate follows a cosine schedule with a linear warmup over the first 10% of training steps. Table 4.2 gives the complete training configuration. The router achieves 100% per-primitive binary accuracy on the held-out validation set across all four primitives by epoch 7 (approximately 750 gradient steps), and training continues to epoch 30 to minimise the cross-entropy further, reaching a final value of 0.0261.

Table 4.2: FSM router training configuration.

Hyperparameter	**Value**
Total expressions	8000
Training split	85% (6800)
Validation split	15% (1200)
Epochs	30
Batch size	64
Optimiser	AdamW
Learning rate	3×10^{-3}
Weight decay	10^{-4}
Gradient clip norm	1.0
LR schedule	Cosine with 10% linear warmup
Final loss	0.0261
Final primitive accuracy	100% (all four primitives)

The expression templates used during training span a range of surface forms for each operation (for example, `{a} + {b}`, `{a} plus {b}`, `compute {a} + {b}`), as well as mixed expressions containing multiple primitives. Operands are drawn uniformly from $[1, 999]$. Because the input to the router is the predicate sequence rather than raw token IDs, the trained model generalises immediately to any operand magnitude.

4.2.3 Predicate Selectivity and Natural-Language Robustness

A potential concern for prompts that mix natural language with arithmetic is whether the router fires spuriously on incidental numeric tokens. The sentence *I have 3 apples and 5 oranges* contains two numeric tokens and the word *and*, which is not in the keyword map and therefore maps to `OTHER`. The resulting predicate sequence is `NUMBER OTHER OTHER NUMBER`, which never contains a `PLUS` predicate. Because the addition FSM is initialised with a strong identity bias and trained exclusively on expressions of the form `NUMBER PLUS NUMBER`, the `PLUS` predicate is the load-bearing gating condition: without it, the FSM remains in its intermediate state and the readout stays near zero throughout.

This property holds by construction of the predicate vocabulary in `predicates.py`. Only explicit operator symbols (`+`, `*`, `%`, and so on) and their written equivalents (*plus*, *times*, *mod*) are mapped to operator predicates. Words that are semantically suggestive of an operation but not part of the keyword map (*and*, *with*, *more than*) resolve to `OTHER` and are invisible to the FSM transitions.

We verify this empirically by running the trained router on several probe inputs.

Table 4.3: Router addition activation ($A_{t,\mathrm{add}}$) at each token for representative probe inputs. The router fires only when an explicit operator predicate appears between two numeric tokens.

Input	**Token at peak**	**Peak $A_{t,\mathbf{add}}$**
`calc: 36+59=`	`59` (after `+`)	0.906
`I have 3 apples and 5 oranges`	none	0.014
`I have 3 + 5 things`	`5` (after `+`)	0.906
`3 things and 5 more`	none	0.014

The router reaches 0.906 only when an explicit `+` symbol is present between two numeric tokens, regardless of surrounding natural-language context. Prompts containing two numbers with no operator predicate between them produce activations indistinguishable from the baseline near-zero level (0.014), confirming that the FSM learned the strict pattern `NUMBER PLUS NUMBER`, not a looser co-occurrence of numerals. In a mixed prompt such as *The answer to 3 apples and 5 oranges is calc: 3+5=*, the router ignores the natural-language numbers entirely and fires only at the calculator expression.

4.3 Steering Vector Extraction

4.3.1 Anchor Position

For each input prompt, the FSM router maps the predicate sequence derived from the Qwen tokenisation to a per-position activation vector. The *anchor position* t^* is the last token index of the predicate group at which $A_{t,k}$ is maximised for the target primitive k, subject to $\max_t A_{t,k} \geq \tau_{\text{FSM}} = 0.5$. This position corresponds to the point at which the complete pattern has been recognised, and is therefore the natural site at which to extract a representation of the pending computation and also the site at which steering injection will be applied at inference time. Anchoring extraction and injection at the same position ensures that the injected vector lies in the same distributional neighbourhood as the activations from which it was computed. This consistency is necessary because residual-stream representations are strongly position-dependent: a vector extracted at one token position carries contextual information specific to that position, and injecting it at a different position would place it out of distribution with respect to the activations already present there, undermining the causal interpretation of the intervention.

4.3.2 Contrastive Extraction

For each primitive k, we generate $N = 3000$ single-primitive prompts of the form `calc: a OP b=` and run the model greedily to obtain logits for the first output token. In place of binary correctness, we compute a log-probability margin as the bucketing signal for each prompt,

$$m_i = \log P(y_1^* \mid x_i) - \max_{y \neq y_1^*} \log P(y \mid x_i)\,, \tag{4.4}$$

where y_1^* is the correct first output token and x_i is the i-th prompt. A positive margin indicates that the model assigns higher probability to the correct token than to any alternative; a strongly negative margin indicates near-certain failure. This continuous signal is preferable to binary correctness for primitives on which baseline accuracy is low, because it allows all 3000 prompts to contribute to the extraction rather than only the small fraction on which the model happens to succeed.

Prompts are ranked by m_i. The top quartile ($m_i \geq \tau_{75}$, the 75th percentile) forms the high-confidence set and the bottom quartile ($m_i \leq \tau_{25}$, the 25th percentile) forms the low-confidence set; the middle 50% is discarded to maximise the contrast between the two groups. For each layer $l \in \{0, \ldots, 35\}$ and each primitive k, residual stream activations at the anchor token t^* are collected from up to $n_{\text{bucket}} = 200$ prompts in each set. The

per-layer steering vector is their mean difference,

$$v_{k,l} \;=\; \mathbb{E}[\, h_{l,t^*} \mid m_i \geq \tau_{75} \,] \;-\; \mathbb{E}[\, h_{l,t^*} \mid m_i \leq \tau_{25} \,]\,. \tag{4.5}$$

Steering vectors are extracted independently for all 36 transformer layers of `Qwen3-4B`, leaving the identification of the most effective layer to a subsequent evaluation sweep. The operand range for addition is drawn from $[1, 999]$, spanning one-, two- and three-digit numbers, so that the extracted direction captures operation type and not the magnitude regime of the operands. Table 4.4 summarises the full extraction configuration.

Table 4.4: Steering vector extraction configuration. The log-probability margin (Eq. 4.4) is used as the bucketing signal for all primitives.

Parameter	**Value**
Prompts per primitive	3000
Bucketing metric	Log-probability margin
High-confidence threshold	75th percentile of m_i
Low-confidence threshold	25th percentile of m_i
Max samples per bucket	200
FSM firing threshold	0.5
Layers extracted	0–35 (all 36)
Model precision	bfloat16
Operand ranges by primitive	
Addition	$a,\, b \;\in\; [1,\, 999]$
Subtraction	$a,\, b \;\in\; [100,\, 999],\;\; a \geq b$
Multiplication	$a,\, b \;\in\; [2,\, 99]$
Modular	$a \;\in\; [100,\, 999],\;\; b \;\in\; [2,\, 13]$

4.4 Local Steering at Inference Time

At inference time, the router is run on the tokenised prompt to produce an activation matrix $A_{\text{tok}} \in \mathbb{R}^{T \times K}$, where each token inherits the router score of its enclosing predicate group. Forward hooks registered at each selected layer l modify the residual stream by

$$h_{l,t} \;\leftarrow\; h_{l,t} + \alpha \sum_{k=1}^{K} A_{\text{tok},\,t,\,k} \cdot v_{k,l}, \tag{4.6}$$

where $\alpha > 0$ is a scalar amplitude and the sum is over all K active primitives. The gate $A_{\text{tok},\,t,\,k} \in [0, 1]$ ensures that the magnitude of the perturbation at position t is proportional to the router's confidence that primitive k is active there. Tokens inside the recognised expression receive the steering signal scaled by the activation; tokens outside it receive negligible contributions when the router fires selectively. Only prompt tokens are perturbed, as tokens generated autoregressively were not present when the predicate

sequence was computed and therefore receive no injection. This design allows multiple arithmetic primitives to coexist within a single prompt without requiring an explicit segmentation, since each primitive's vector is gated independently by the corresponding SoftFSM output.

A *global* ablation mode applies each $v_{k,l}$ uniformly at all token positions and with all primitives summed, without any position-dependent gating. Comparing the local and global modes isolates the contribution of the router's positional selectivity.

4.4.1 Evaluation Protocol

The evaluation sweeps over all layers for which steering vectors have been extracted and over amplitudes $\alpha \in \{0.5, 1.0, 2.0, 5.0\}$ on a held-out set of 300 prompts per primitive. Three metrics are computed relative to an un-steered baseline. *First-token accuracy* measures whether the first generated token matches the correct answer. *Full-answer accuracy* measures whether the entire generated string is correct. *Digit accuracy* measures the fraction of digit characters in the generated string that match the corresponding character in the ground-truth answer, providing a more sensitive signal for multi-digit outputs where a model that recovers the leading digit but fails on later ones scores zero on full-answer accuracy but non-zero on digit accuracy.

An optional out-of-distribution evaluation resamples operands from a range not used during extraction (for example, four-digit operands in $[1000, 9999]$), providing a direct test of whether the steering direction transfers across magnitude regimes. The baseline and steered evaluations are conducted identically, with all metrics reported as absolute values and signed deltas relative to the unsteered model.

4.5 Analysis of Extracted Steering Vectors

Before evaluating the causal effect of steering on model accuracy, we examine the norms $\|v_{k,l}\|$ of the extracted vectors across all 36 layers and all four primitives. The norm at layer l for primitive k measures the Euclidean separation between the high- and low-confidence residual-stream clusters at that depth. Figure 4.2 plots this quantity at every layer.

The norms grow monotonically with depth for every primitive, but this trajectory is primarily a scale artefact. Residual-stream vectors accumulate magnitude through skip connections at every layer. As a result, $\|v_{k,l}\|$ at a given depth reflects the geometric scale of activations at that point rather than the informativeness of the corresponding direction. A large norm at layer 35 does not imply that layer 35 is the most effective injection site.

More informative than the overall trend is a consistent phase transition at layers 6 through 8 appearing across all four primitives. Below layer 6 norms remain small and grow slowly; between layers 6 and 8 they increase by a factor of two to three over just two

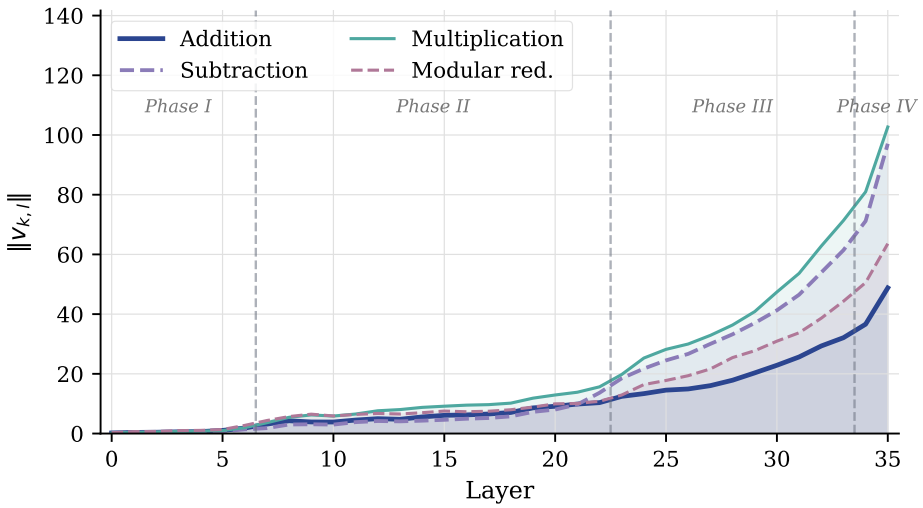


Figure 4.2: Steering vector norms $\|v_{k,l}\|$ at every layer for all four primitives. Phase boundaries (dashed lines) are placed at layers 5–6, 19–20, and 33–34, derived from the norm trajectories of these primitives. Phase I (layers 0–5): slow initial encoding. Phase II (layers 6–19): active separation regime, beginning with a rapid norm increase at layers 6–8 followed by moderate growth. Phase III (layers 20–33): accelerating scale accumulation with increasing inter-primitive divergence. Phase IV (layers 34–35): sharp final uptick visible across all primitives.

consecutive layers. This abrupt separation suggests that layers 6 through 8 are the earliest depth at which computation-mode information becomes linearly separable. The finding aligns with prior mechanistic analyses of small arithmetic transformers, which consistently locate algorithmic processing in early-to-middle layers [**?** **?**].

Immediately following this transition, a mild but consistent local dip appears at layers 9 and 10 across all four primitives. The norm retreats slightly before resuming its upward trend. This pattern is too consistent across primitives to be coincidental, and it plausibly marks the point at which one functional circuit completes its contribution and transfers control to the next.

Taken together, the observations suggest that computation-mode information is most distinctively encoded in roughly layers 8 through 20. Below this range the phase transition has not yet separated the clusters; above it, scale inflation begins to dominate absolute norms. This interval is the most natural target for steering, and the evaluation sweep confirms which specific layer and amplitude yields the largest accuracy gain.

Chapter 5

Concept Localisation via Contrastive Residual-Stream Analysis

The extraction procedure described in Chapter 4 partitions prompts by their log-probability margin. This signal reflects aggregate computational confidence rather than the state of any specific sub-operation within the arithmetic pipeline. The resulting steering vectors capture the mean difference between high- and low-confidence residual-stream distributions without isolating the contribution of individual sub-computations such as carry. Whether carry is encoded as a distinct, localised direction, and at what network depth this encoding first emerges, requires a more controlled experimental design than the confidence-based bucketing can provide.

This chapter investigates that question through a contrastive analysis in which prompt pairs are matched so that the only systematically varying factor is whether a units-column carry occurs. The methodology draws on representation engineering [**?**] and on contrastive activation addition [**?**], both of which extract concept directions by taking mean differences in activation space across semantically contrasting inputs. Here the technique serves as a diagnostic for concept localisation, using the layer-wise norm profile of the resulting delta vectors to characterise where the carry representation crystallises.

Carry is a natural target for this analysis because it is a binary, locally determined event. Given the units digits of both operands, whether a carry propagates is fixed. **?**] identified a dedicated carry-propagation circuit in a small transformer trained on integer addition, establishing that carry can be implemented as a localised, mechanistically distinct sub-computation. Whether an analogous structure exists in `Qwen3-4B` is an open question. A large instruction-tuned model receives no explicit arithmetic curriculum, and finding a localised carry representation would suggest that mechanistic modularity is a general organisational principle rather than an artefact of task-specific training.

5.1 Contrastive Pair Construction

A carry pair consists of two prompts sharing the same first operand a and all but the units digit of the second operand b. Letting $a_0 = a \bmod 10$, the carry variant uses b^+ with $a_0 + (b^+ \bmod 10) \geq 10$, while the no-carry variant uses b^- with $a_0 + (b^- \bmod 10) < 10$. The two prompts produce token sequences of equal length differing at exactly one position, the units digit of b.

To confine the signal to a single carry event and exclude cascading carries into the tens column, we impose the constraint

$$\Big(\lfloor a/10 \rfloor \bmod 10\Big) + \Big(\lfloor b/10 \rfloor \bmod 10\Big) \;\leq\; 8, \tag{5.1}$$

so that the tens-column sum, even after receiving a units carry of 1, remains strictly below ten. Under this constraint the carry and no-carry prompts differ in exactly one computational event. The mean difference in residual-stream activations across many such pairs is therefore attributable to that single event rather than to higher-order carry interactions.

The anchor position t^* is identified by comparing the two tokenisations and taking the last index at which they differ. Because `Qwen3` represents each decimal digit as a single token, this corresponds to the units digit of b without exception, requiring no auxiliary alignment step. It is the earliest position at which the model holds the units digits of both operands in context and can in principle determine whether a carry will occur.

Pairs are generated for three prompt templates, T0 (`calc: {a}+{b}=`), T1 (`calc: {a} + {b} =`), and T2 (`What is {a}+{b}? Answer:`). The templates differ in whitespace and surface phrasing. Repeating the extraction across all three provides a check on whether the extracted direction reflects the model's carry representation or a pattern tied to one specific tokenisation. Table 5.1 summarises the configuration.

Table 5.1: Concept localisation experimental configuration.

Parameter	**Value**
Pairs per template	200
Templates	T0, T1, T2
Operand digits	3
No-cascade constraint	$\lfloor a/10 \rfloor \bmod 10 + \lfloor b/10 \rfloor \bmod 10 \leq 8$
Anchor detection	Last differing token position
Layers extracted	0–35 (all 36)
Model precision	bfloat16
Top features reported per layer	15

5.2 Layer-Wise Concept Deltas

Let $\mathcal{D}_+$ and $\mathcal{D}_-$ denote the sets of carry and no-carry prompts respectively. For each layer $l \in \{0, \ldots, L-1\}$, the carry concept direction is defined as the mean difference of residual-stream vectors at the anchor position,

$$\delta_l \;=\; \mathbb{E}_{x \in \mathcal{D}_+}[h_{l,t^*}(x)] \;-\; \mathbb{E}_{x \in \mathcal{D}_-}[h_{l,t^*}(x)]\,, \tag{5.2}$$

where $h_{l,t}(x) \in \mathbb{R}^d$ is the residual-stream vector at layer l and position t for prompt x, consistent with the notation of Chapter 4. Under the linear representation hypothesis [**?**], δ_l approximates the axis along which carry is encoded at depth l, provided the concept is linearly separable and the matched-pair averaging suppresses nuisance variation. The construction in Section 5.1 is designed to make that nuisance small. d The relationship between δ_l and the steering vectors $v_{k,l}$ of Chapter 4 is instructive. Both are mean-difference vectors in the residual stream, but they partition the prompt distribution along different axes. The steering vectors $v_{k,l}$ separate prompts by model confidence, capturing the direction that discriminates successful from unsuccessful computation. The concept deltas δ_l separate prompts by the presence or absence of a specific sub-operation, independently of whether the model ultimately produces the correct answer. The former is operationally motivated; the latter is conceptually motivated. Only the second admits a localisation analysis that is not confounded by aggregate performance.

5.3 Sharpness and Concept Localisation

The sequence $\left\{\|\delta_l\|\right\}_{l=0}^{L-1}$ constitutes a norm trajectory tracking how the representational distance between carry and no-carry conditions evolves with depth. A steep rise at a single layer followed by a plateau suggests the carry concept crystallises there. Gradual growth throughout suggests a distributed encoding with no dominant localisation site.

To quantify localisation, we define the sharpness index as the fraction of total norm mass concentrated within a window of width three centred at the peak layer $l^* = \arg\max_l \|\delta_l\|$,

$$\psi \;=\; \frac{\displaystyle\sum_{l=\max(0,\,l^*-1)}^{\min(L-1,\,l^*+1)} \|\delta_l\|}{\displaystyle\sum_{l=0}^{L-1} \|\delta_l\|}. \tag{5.3}$$

A value of ψ close to one indicates tight localisation around a single layer. A value close to $3/L$ indicates that the norm is approximately uniformly distributed with depth. The index thus provides a single scalar summary of the degree to which carry is encoded at a

specific processing stage.

Complementary to the norm trajectory, the inter-layer alignment $\rho_l = \cos(\delta_l, \delta_{l+1})$ measures whether the concept direction rotates between consecutive layers. Consistently high alignment would indicate a single direction that merely scales with depth. A sharp drop at a particular transition marks a qualitative reorientation, suggesting the carry concept is being re-encoded in a different subspace at that depth.

5.4 Feature-Space Interpretation

In the dictionary-learning framework applied to transformer internals, a *feature* is a learned direction $\phi_f \in \mathbb{R}^d$ hypothesised to correspond to a single interpretable concept or computation, where d is the residual-stream width of the transformer. Because transformers represent more independent concepts than they have dimensions, they are believed to superpose many approximately orthogonal feature directions within the residual stream, with each activation being a sparse signed combination of active features [**?**]. A transcoder operationalises this for MLP sub-layers as follows: an encoder $W_l^{\text{enc}} \in \mathbb{R}^{F_l \times d}$ maps the pre-MLP residual-stream vector to a sparse bottleneck of feature activations, and a decoder $W_l^{\text{dec}} \in \mathbb{R}^{d \times F_l}$ reconstructs the MLP output as a sum of active feature effects. Each encoder row $(W_l^{\text{enc}})_f$ is therefore a *detection direction* in $\mathbb{R}^d$.

The delta vectors δ_l are dense in $\mathbb{R}^d$ and do not directly identify which model components encode carry at each layer. The transcoder architecture provides a more interpretable way to decompose them sparsely. Each transcoder at layer l learns an encoder weight matrix $W_l^{\text{enc}} \in \mathbb{R}^{F_l \times d}$, where F_l is the number of learned features and d the residual-stream dimension. The f-th row $(W_l^{\text{enc}})_f$ is the direction to which feature f responds most strongly.

The cosine alignment between the carry delta and each encoder direction,

$$c_{l,f} \;=\; \frac{(W_l^{\text{enc}})_f \cdot \delta_l}{\|(W_l^{\text{enc}})_f\| \cdot \|\delta_l\|}, \tag{5.4}$$

identifies the transcoder features most aligned with the concept delta at that layer. Features with $|c_{l,f}|$ close to one are the monosemantic components whose activation pattern best accounts for the carry versus no-carry difference. This projection connects the dense, direction-based view of representation engineering to the sparse, feature-based view of dictionary learning. The same concept direction can thus be understood both as a point in residual-stream space and as a weighted combination of interpretable transcoder features, bridging two perspectives that the broader interpretability programme has pursued in parallel.

5.5 Template Robustness

Per-template deltas $\delta_l^{(T)}$ are computed separately for each template $T \in \{T0, T1, T2\}$, and their pairwise cosine similarities are measured at each layer. This provides a direct check on whether the extracted direction is a stable property of the model's carry representation. Consistently high cross-template alignment at the layers where $\|\delta_l\|$ peaks would confirm that the same direction emerges regardless of surface form. Divergence at early layers would indicate that template-specific variation persists until mid-network processing assimilates the input into a common representational format, providing additional evidence about the depth at which template-invariant concept encoding begins. This consistency check is the analogue, at the level of individual sub-operations, of the cross-template steering evaluation in Chapter 4.

5.6 Causal Validation

The delta vectors and sharpness metrics of the preceding sections characterise the *representational* structure of each concept, quantifying where a separable direction exists in residual-stream space. They do not, however, establish that the extracted representation is causally upstream of the model's output. A direction may be strongly present at a given layer without being mechanistically connected to the prediction. Two complementary interventions test this causal claim directly, and are applied to every concept in the suite.

5.6.1 Activation Patching

Activation patching [**?**] isolates the causal contribution of a specific layer by replacing a single intermediate representation and measuring the resulting change in output. For each contrastive pair (x^+, x^-) and each layer l, the positive prompt's residual-stream vector at the anchor position t^* is transplanted into the negative prompt's forward pass. The score is the induced change in the logit margin at the final prediction position,

$$S_l^{\text{patch}} = \Big[f_{\text{pos}}\big(\tilde{x}_{[l]}^-\big) - f_{\text{neg}}\big(\tilde{x}_{[l]}^-\big)\Big] - \Big[f_{\text{pos}}\big(x^-\big) - f_{\text{neg}}\big(x^-\big)\Big], \tag{5.5}$$

where $\tilde{x}_{[l]}^-$ denotes the negative prompt with $h_{l,t^*}(x^-)$ replaced by $h_{l,t^*}(x^+)$, and f_{pos}, f_{neg} are the logits for the positive and negative answer tokens at the last position. A large positive S_l^{patch} indicates that the layer-l concept representation, when inserted into the negative context, is sufficient to shift the model's output toward the positive answer. The metric therefore directly quantifies the *causal sufficiency* of the layer-l encoding for driving the correct response.

5.6.2 Gradient Dot Delta

Activation patching requires a full forward pass for each patched layer, making it the more expensive of the two methods. A cheaper first-order approximation is obtained by differentiating the output margin with respect to the residual stream at the negative prompt's operating point. For each pair and each layer l, a single forward pass on x^- is followed by a single backward pass to compute the gradient of the margin with respect to $h_{l,t^*}(x^-)$. The score is the inner product of this gradient with the precomputed concept delta,

$$S_l^{\text{grad}} \;=\; \nabla_{h_{l,t^*}} m\Big(x^-\Big) \cdot \delta_l, \tag{5.6}$$

where $m(x) = f_{\text{pos}}(x) - f_{\text{neg}}(x)$ is the logit margin evaluated at prompt x, and the gradient is computed at the negative prompt's residual stream.

The relationship between S_l^{grad} and S_l^{patch} is made precise by a first-order Taylor expansion of the patched margin around the unpatched activation,

$$m\Big(\tilde{x}^-_{[l]}\Big) \;\approx\; m\Big(x^-\Big) + \nabla_{h_{l,t^*}} m\Big(x^-\Big) \cdot \Big(h_{l,t^*}(x^+) - h_{l,t^*}(x^-)\Big). \tag{5.7}$$

Taking the expectation over pairs and using $\delta_l = \mathbb{E}[h_{l,t^*}(x^+)] - \mathbb{E}[h_{l,t^*}(x^-)]$ gives $\mathbb{E}[S_l^{\text{patch}}] \approx S_l^{\text{grad}}$ to first order, provided the gradient varies little across the pair distribution. The gradient-dot-delta score is therefore the linearised, single-pass analogue of activation patching; agreement between the two methods at any given layer constitutes evidence that the output function is approximately linear in the relevant neighbourhood of residual-stream space. Systematic divergence indicates nonlinearity or higher-order interactions that the gradient cannot capture. Both methods are anchored at the same position t^* used for delta extraction throughout this chapter.

The contrastive methodology described in the preceding sections is not specific to carry. To test whether concept localisation generalises across qualitatively different domains, the same pipeline was applied to sixteen additional concepts spanning three categories. The *mathematics* category contains concepts that are binary and locally determined by small integer inputs. The *physics* category contains concepts whose positive instance satisfies a physical law or conservation principle. The *logic and reasoning* category contains concepts requiring logical inference or structural consistency across multiple constituents.

Figure **??** shows a representative pair for each of the seventeen concepts. The positive instance appears above the negative instance, with **bold** tokens marking the single point of difference between them. Because exactly one or two tokens change within each pair, the resulting delta vector is attributable to the concept alone and not to differences in token count, operand magnitude, or surface phrasing. Each concept is evaluated under three template forms: T0 uses compact symbolic notation, T1 a natural-language description, and T2 an interrogative phrasing.

The modular arithmetic concepts share a structural property with carry: the binary outcome is determined by a single arithmetic relation between small integers, and the concept is expressible as a test against a fixed modulus. The relational concepts require a different kind of evaluation. Transitive ordering and negation scope involve logical consistency across two or three values; energy conservation involves a physical plausibility judgment; causal direction involves the assignment of asymmetric roles to two entities. These distinctions motivate asking whether the two groups produce qualitatively different norm trajectories and whether their representations crystallise at similar depths.

Figure **??** shows the layer-wise delta norm for all seven concepts on a shared axis. Modular arithmetic concepts are drawn as solid lines; relational concepts as dashed lines. The right panel normalises each curve by its maximum value so that the growth profiles are comparable across concepts with very different absolute magnitudes.

5.7 Results

5.7.1 Norm Trajectory and Phase Structure

Figure **??** shows the layer-wise norm profile of the carry delta vector and the pairwise cross-template consistency on a shared axis, with three processing phases annotated. The norm trajectory reveals a clear three-stage structure that maps directly onto qualitatively distinct computational regimes.

In Phase I (layers 0–4), the norm rises slowly from 1.7 to 2.4 as the model processes the raw token embeddings without yet forming a strong carry-specific representation. The first transition at layers 4–5 marks the entry into Phase II (layers 5–18), where the norm jumps to 6.0 and plateaus around 8–12: carry information crystallises here into a stable intermediate representation. A second transition at layer 18–19 raises the norm abruptly to 16.5, opening Phase III (layers 19–35), in which the representation grows steadily to its peak of 38.0 at layer 34 before dropping to 34.75 at the final layer. This terminal drop is consistent with the accumulated carry direction being partially projected into output-logit space at the last transformer block.

The sharpness index $\psi = 0.182$ quantifies the degree of localisation. For a 36-layer network, a uniform distribution would give $3/36 \approx 0.083$; the observed value is approximately twice this baseline, indicating that carry is a progressively enriched distributed representation rather than a sharply localised one, with growth concentrated at the two phase transitions.

5.7.2 Representational Geometry: Cross-Layer Similarity

To characterise how the carry direction evolves in representational geometry, Figure **??** shows the full 36×36 matrix of pairwise cosine similarities between delta vectors at all layer combinations.

The matrix reveals a clear block-diagonal structure aligned with the three phases. Within each phase, cosine similarities are high (above 0.90 on the diagonal band), confirming that the carry direction is stable and approximately non-rotating within a processing stage. Across phases, similarities are lower, particularly between Phase I and Phase III where the off-diagonal blocks approach 0.70–0.75. This quantifies the degree to which the early and late carry representations, while pointing in the same broad direction, differ meaningfully in subspace orientation. The carry concept is not re-encoded from scratch at each transition but is continuously refined, with the phase boundaries marking the most significant directional shifts. This geometry is consistent with the inter-layer alignment ρ_l remaining above 0.88 everywhere while still producing substantial cumulative rotation across the full depth of the network.

5.7.3 Template Robustness and Output Preparation

The bottom panel of Figure **??** shows cross-template consistency throughout the network. All three template pairs remain above 0.97 through layer 24, confirming that the extracted carry direction is a genuine property of the model's internal computation rather than an artefact of any particular surface form. The same direction emerges independently of whether the operands are presented in calculator syntax or natural language.

Beyond layer 24 a divergence emerges specifically between the natural-language template T2 and the two calculator-format templates T0 and T1 (which differ only in whitespace and stay aligned at 0.94 through layer 35). By layer 35, T1 vs T2 falls to 0.78. The divergence is driven not by a difference in carry computation itself but by how the late layers specialise the representation for the expected output format: T2 elicits a carry direction that begins to rotate toward token probabilities appropriate for natural-language continuation rather than a numeric calculator response.

This establishes a clean separation between two regimes within Phase III. In layers 19–24, the representation is still template-invariant and conceptually interpretable. From approximately layer 25 onward, it becomes output-specific. The transition at layer 25 thus marks the boundary between *concept encoding* and *output preparation* in the model's arithmetic processing pipeline, a distinction that has practical implications for where interventions such as steering vectors are most likely to act on the underlying computation rather than on output formatting.

5.7.4 Feature-Space Decomposition

Figure **??** shows the top-15 transcoder features most aligned with the carry delta δ_l at each layer. Each point represents a feature f at layer l, with colour encoding the sign of $c_{l,f}$ (red positive, blue negative) and dot area proportional to $|c_{l,f}|^{1.5}$. The bottom strip repeats the carry delta norm for reference.

Across all 36 layers, the 540 selected feature–layer pairs draw from 539 distinct feature identifiers. Only one feature, F73141, appears in the top-15 at more than one layer (layers 21 and 28), a persistence rate below 0.2 per cent of the full feature vocabulary of approximately 160,000 components. This near-complete non-overlap is a strong mechanistic result: carry is not encoded through a small, reused set of monosemantic components, but through a layer-specific sparse basis in which each depth selects a largely disjoint subset of the transcoder dictionary.

The finding is consistent with the phase structure and cross-layer geometry described above. The block-diagonal similarity matrix in Figure **??** shows that within-phase directional alignment is high (above 0.90), predicting that features should be more stable within a phase than across phase transitions. The scatter confirms this qualitatively: visible banding in feature identity is apparent within each phase region, with the sharpest changeovers at the two transition boundaries.

The result also informs the scope of feature-level intervention. If carry were implemented through a small set of persistent features, a transcoder-based surgical edit targeting those features would be a natural next step. The layer-specific basis implies instead that any feature-level intervention would need to target a different subset at each depth. This reinforces the conclusion of Chapter 4 that computation-mode layer selection, rather than individual feature targeting, is the more tractable lever for influencing carry processing in a large pre-trained model.

5.7.5 Causal Validation

Figure **??** shows the activation patching scores S_l^{patch} and gradient-dot-delta scores S_l^{grad} across all 36 layers for all concepts in the suite, with the delta norm of each concept overlaid for reference. The two curves are in close agreement for every concept, confirming that the output margin is approximately linear in the extracted concept direction and that the gradient-dot-delta approximation is reliable in this setting.

Across all concepts, both scores are near zero in the early layers, where representational delta norms are also small, and rise sharply at the layer where each concept's norm profile first stabilises. The peak causal influence consistently precedes the norm peak at late layers, reinforcing the interpretation that late-network growth in $\|\delta_l\|$ reflects output preparation rather than an increase in computational relevance. For carry specifically, the scores decay

toward zero across Phase III in a pattern that mirrors the steering effectiveness results of Section **??**, providing a gradient-based corroboration of the phase boundary identified by the contrastive analysis. The broader concept suite shows a qualitatively similar pattern: causal influence peaks earlier than the representational norm, and late-layer causal scores are suppressed relative to their representational magnitude.

5.8 Relation to Steering Effectiveness

The phase structure identified through contrastive activation analysis provides an independent, concept-level explanation for the layer-wise pattern of steering improvement reported in Chapter 4. Figure **??** aligns the three relevant signals on a shared layer axis: the carry concept delta norm $\|\delta_l\|$, the steering vector norms $\|v_{k,l}\|$ for all four arithmetic primitives, and the per-layer improvement in full-answer accuracy produced by the FSM-routed steering intervention at the best amplitude $\alpha = 5.0$.

A naive comparison of the carry norm and the steering improvement curves yields no linear correlation (Pearson $r = -0.32$, Spearman $r = -0.12$, both non-significant), and the sign is informative. The carry norm grows monotonically to its peak at layer 34 precisely as the steering improvement collapses toward zero, so a global linear relationship between the two signals would be mechanistically misleading. The carry direction becoming *larger* in Phase III does not make it a better target for intervention; as the template consistency analysis showed, Phase III is the regime in which the representation is being projected toward output-logit space rather than maintained as a computation-level encoding.

The meaningful correspondence is structural rather than scalar. The three-phase boundary identified in the carry analysis predicts the effective steering window with high fidelity. In Phase I (layers 0–4) the carry representation has not yet crystallised, and the steering improvement is negligible across all amplitudes. The 4–5 transition, the first point at which carry information becomes linearly stable, coincides with the onset of meaningful steering gain. Phase II (layers 5–18) is the regime of stable, template-invariant carry encoding identified in Section 5.5: it is also the regime in which steering improvement rises sharply, peaking at layer 11 with a full-answer accuracy gain of +47.3 percentage points over the unsteered baseline. The 18–19 transition, the entry into Phase III, coincides with the onset of steering degradation: improvement falls continuously thereafter, and by layers 30–35 the intervention is marginally harmful.

Figure **??** makes the bell-shaped layer profile of steering improvement explicit. The peak at layer 11 falls squarely within Phase II, and the return to baseline in Phase III is consistent with the output-preparation hypothesis: injecting a computation-mode steering vector into layers that are already projecting toward output tokens disrupts the formatting of the response without reinforcing the underlying arithmetic operation.

A further observation concerns the steering vector norms shown in the middle panel of Figure ??. The norms for the four primitives are nearly indistinguishable and all grow monotonically, driven by residual-stream scale accumulation rather than by any primitive-specific structure. In particular, addition and subtraction, both subject to carry events, track multiplication and modular, which involve no carry, throughout the network. This confirms that the sv norm alone is an unreliable predictor of steering effectiveness: the relevant factor is the geometric orientation of the extracted direction relative to the computation being performed at that depth, not its Euclidean magnitude.

Taken together, the carry localisation analysis and the steering evaluation are mutually consistent and mutually explanatory. The concept-level analysis predicts which layers are computation-mode layers and which are output-preparation layers; the intervention experiment confirms that only computation-mode layers admit effective steering. The two results are produced by entirely independent pipelines, contrastive activation averaging on matched carry pairs and confidence-bucketed extraction followed by causal injection, and their agreement strengthens the mechanistic interpretation of both.

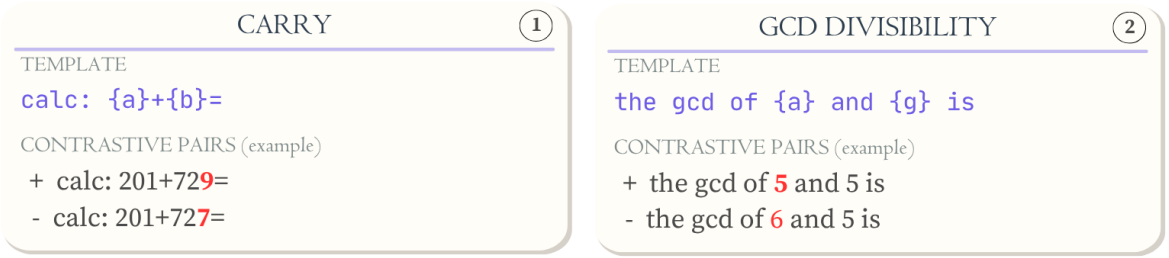


Figure 5.1: Representative contrastive pairs for all seventeen concepts, grouped by domain. Each entry shows the positive instance (top) and negative instance (bottom); tokens that differ between the two are highlighted. Concepts span three categories: mathematics (top), physics (middle), and logic and reasoning (bottom).

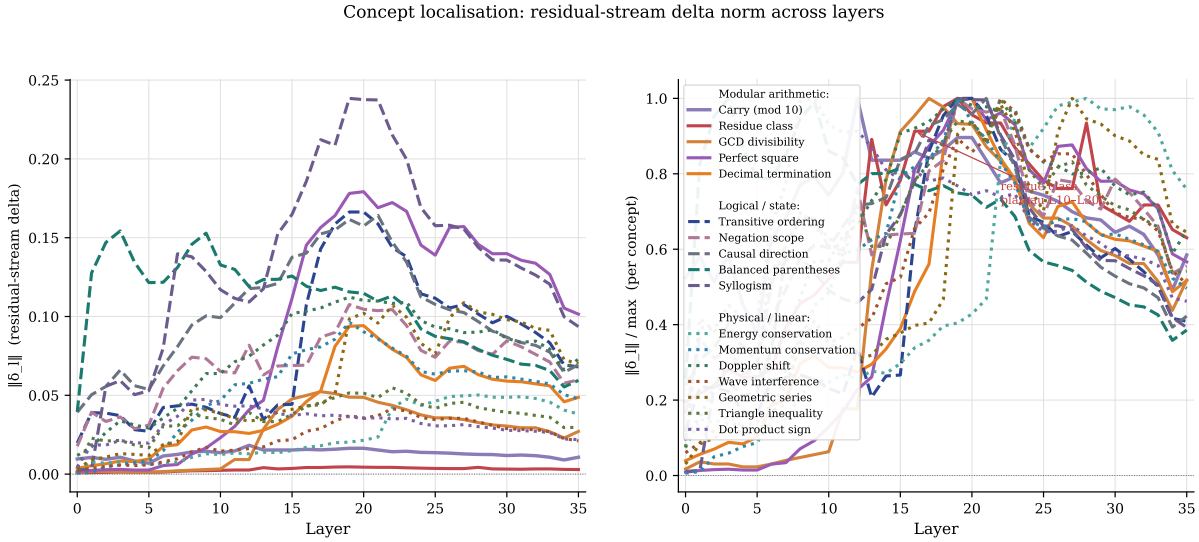


Figure 5.2: Layer-wise delta norm for all seven concepts. Left: raw norms. Right: norms divided by the per-concept maximum, so growth profiles are comparable. Modular arithmetic concepts (carry, GCD divisibility, residue class) are shown as solid lines; relational concepts (transitive ordering, energy conservation, causal direction, negation scope) as dashed lines. Vertical dashed lines mark the phase boundaries identified in the carry analysis.

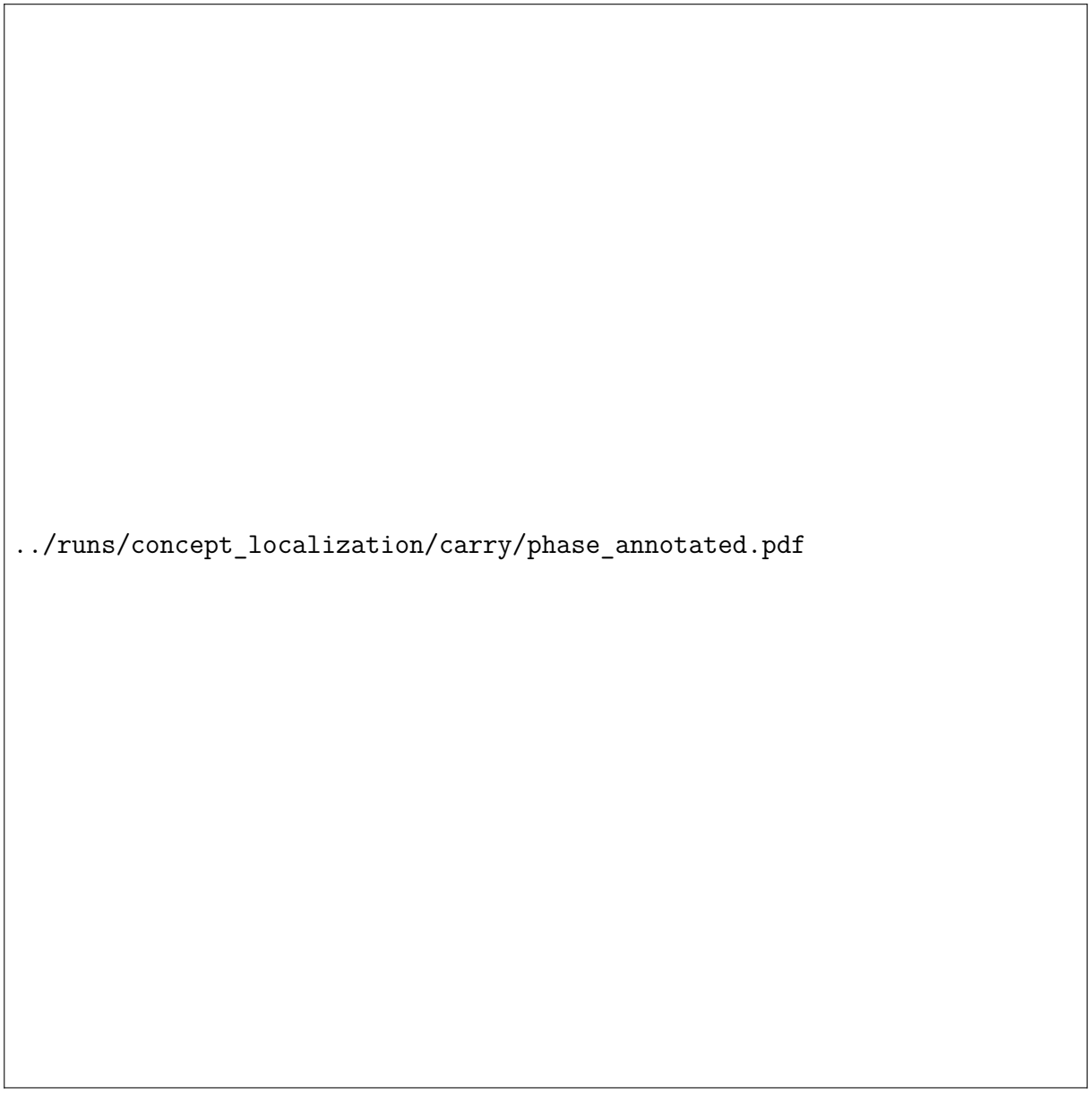


Figure 5.3: Top: norm of the carry concept delta $\|\delta_l\|$ with phase regions annotated and the peak at layer 34 marked. All four curves (pooled and per-template) are nearly indistinguishable, confirming template-invariant encoding throughout most of the network. Bottom: cross-template cosine similarity of the carry direction. The shaded red region marks the late-layer divergence zone where template-specific output preparation begins.

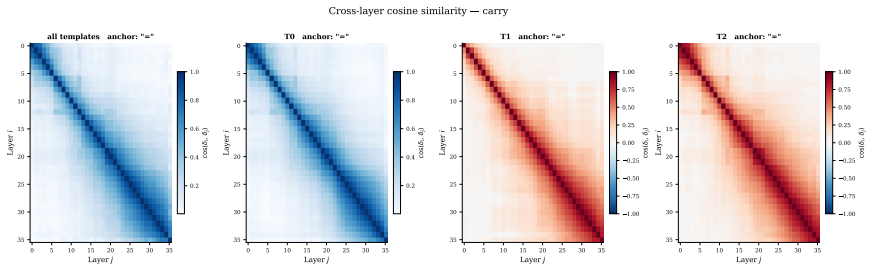
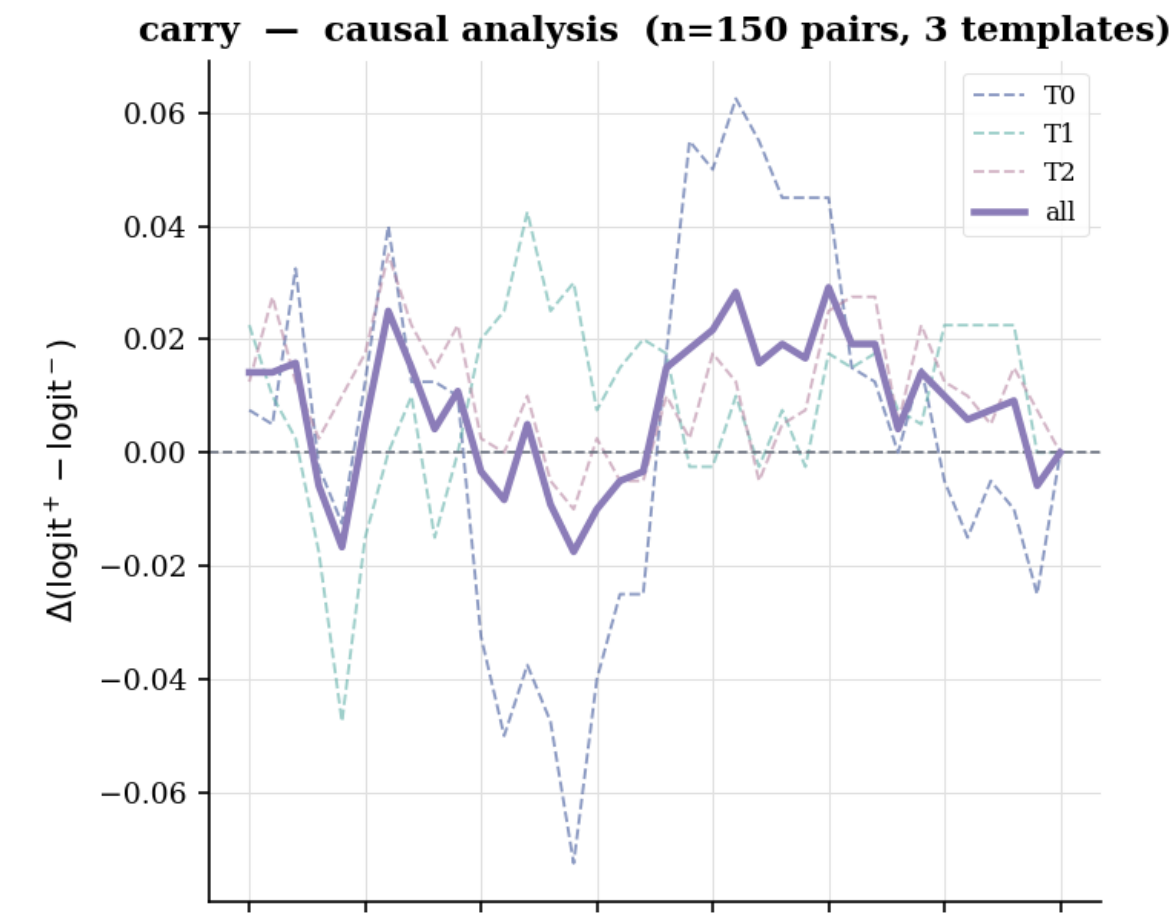
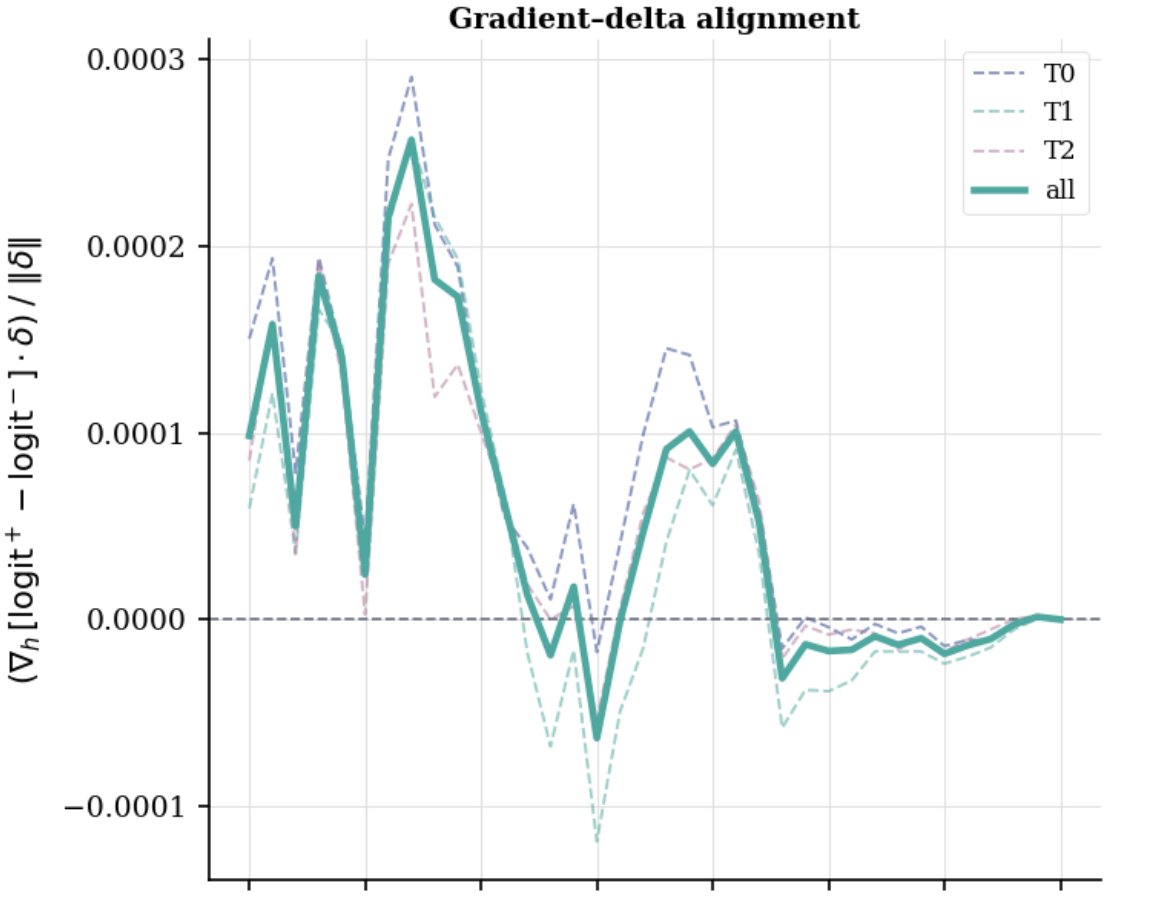
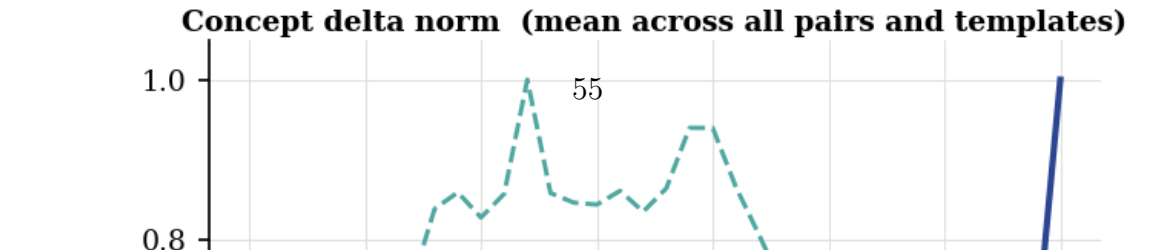


Figure 5.4: Cross-layer cosine similarity matrix $\cos(\delta_i, \delta_j)$ for all layer pairs. Dashed white lines mark the phase boundaries at layers 4–5 and 18–19. The block structure confirms that the carry direction is more similar within phases than across them, and that phases I and III share the least alignment, indicating a qualitative reorientation of the representation between early and late processing stages.

Figure 5.5: Top-15 transcoder features most aligned with the carry concept delta δ_l at each layer (equation 5.4). Dot area is proportional to $|c_{l,f}|^{1.5}$; red denotes positive alignment (feature activates more for carry), blue negative (activates more for no-carry). Phase boundaries are dashed vertical lines. Feature F73141 (outlined dots) is the only feature that recurs in the top-15 at more than one layer, appearing at layers 21 and 28 (annotations above and below the respective points). The bottom strip shows $\|\delta_l\|$ for context. Carry is encoded via a layer-specific sparse basis: the top-aligned features are almost entirely non-overlapping across layers, with no single monosemantic component persisting through the network.


carry — causal analysis (n=150 pairs, 3 templates)
T0
T1
T2
all
0.06
0.04
0.02
0.00
−0.02
−0.04
−0.06
Δ(logit⁺ − logit⁻)


Gradient-delta alignment
T0
T1
T2
all
0.0003
0.0002
0.0001
0.0000
−0.0001
(∇h [logit⁺ − logit⁻] · δ) / ‖δ‖


Concept delta norm (mean across all pairs and templates)
1.0
0.8
55

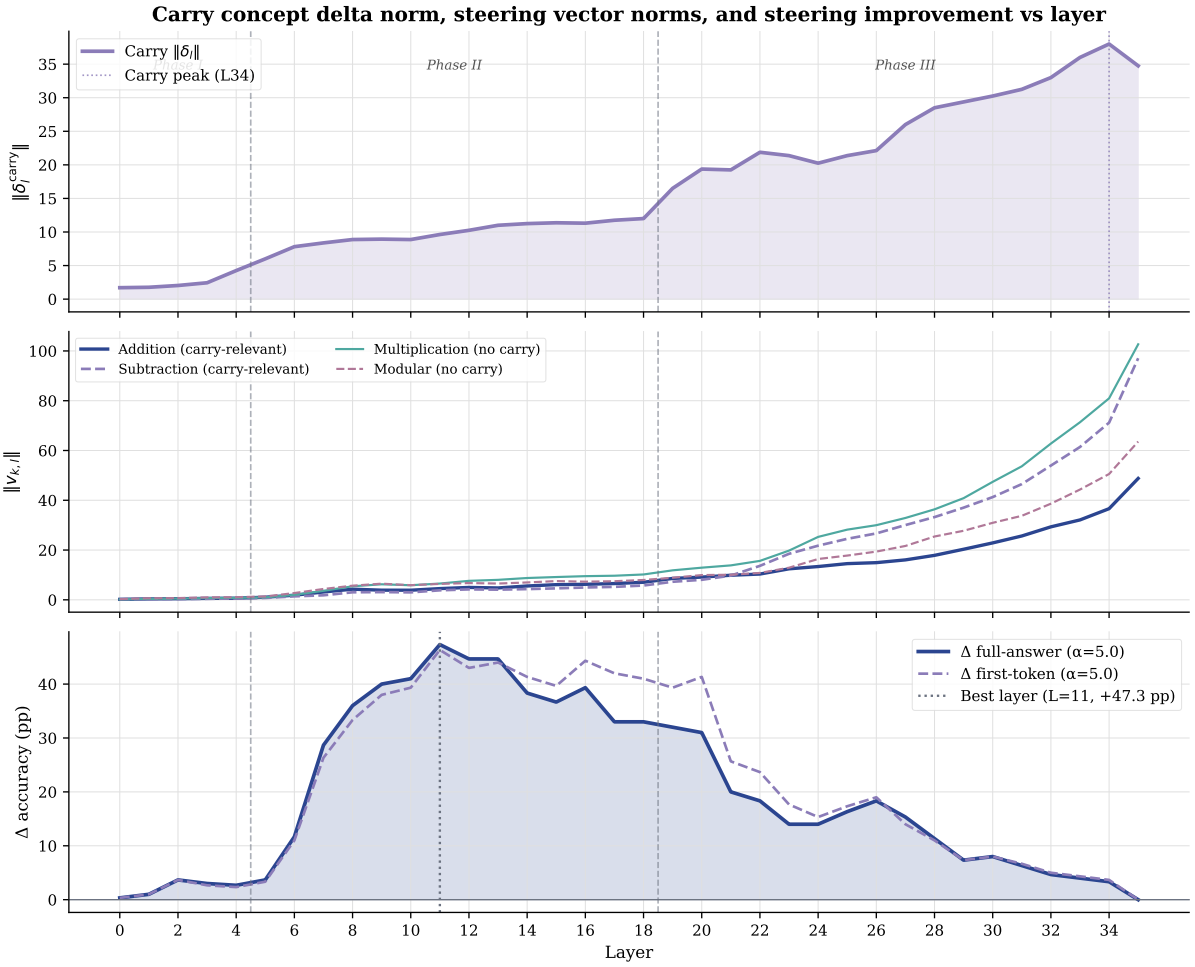


Figure 5.7: Three-panel comparison across all 36 layers. Top: carry concept delta norm $\|\delta_l\|$ (purple) with phase regions shaded and the peak at layer 34 marked. Middle: steering vector norms $\|v_{k,l}\|$ for addition and subtraction (carry-relevant, solid and dashed green) and for multiplication and modular (no carry event, red and orange). Bottom: layer-wise change in full-answer and first-token accuracy produced by local FSM steering at $\alpha = 5.0$, with the optimal injection layer $l^* = 11$ marked. Phase boundaries at layers 4–5 and 18–19 are shown as vertical dashed lines in all panels.

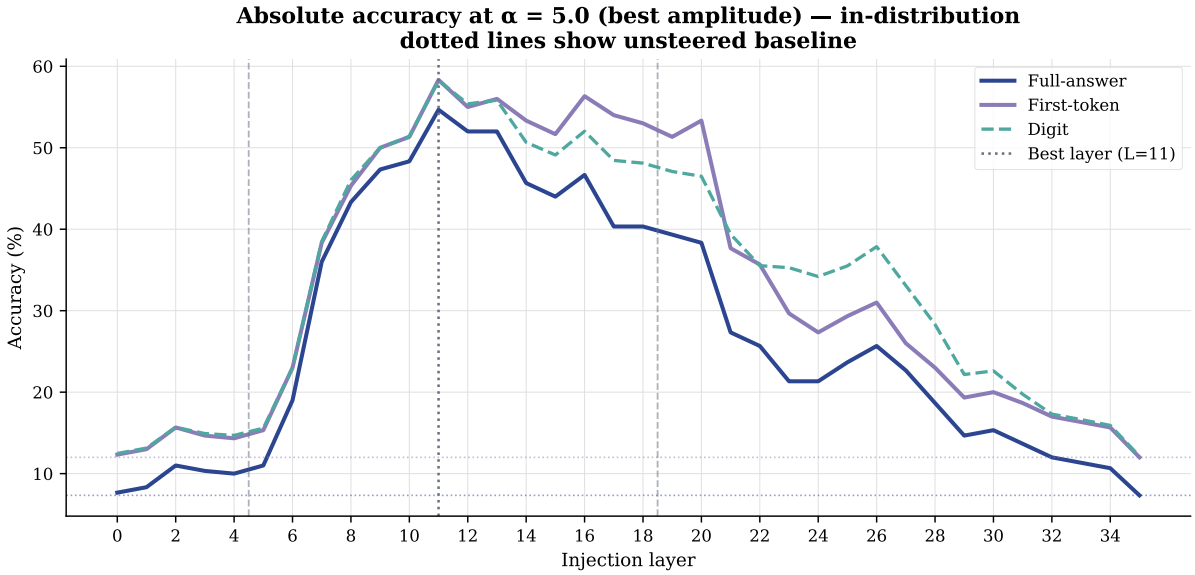


Figure 5.8: Absolute accuracy at the best amplitude $\alpha = 5.0$ as a function of injection layer, for full-answer (green), first-token (blue), and digit (dashed orange) metrics. Dotted horizontal lines show the unsteered baseline for each metric. The bell-shaped profile peaks at layer 11 and falls to near-baseline levels by layer 30, matching the Phase II window of stable carry encoding.

Bibliography

[] Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Zac Hatfield-Dodds, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah E. Burke, Tristan Hume, Shan Carter, Tom Henighan, and Chris Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. URL https://transformer-circuits.pub/2023/monosemanticity/index.html.

[] Arthur Conmy, Augustine N. Mavor-Parker, Aengus Lynch, Stefan Heimersheim, and Adrià Garriga-Alonso. Automated circuit discovery for mechanistic interpretability. In *Advances in Neural Information Processing Systems*, volume 36, 2023. doi: 10.48550/arXiv.2304.14997.

[] Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models, 2023.

[] Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021. URL https://transformer-circuits.pub/2021/framework/index.html.

[] Jack Lindsey, Wes Gurnee, Emmanuel Ameisen, Shan Carter, Tom Henighan, Adam Stevenson, Tom Conerly, Danny Hernandez, Adam Jermyn, Callum McDougall, et al. On the biology of a large language model. 2025. URL https://transformer-circuits.pub/2025/biology-of-a-lm/index.html. Anthropic Transformer Circuits Thread.

[] Tiberiu Musat. Clustering and alignment: Understanding the training dynamics in modular addition, 2024.

[] Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2301.05217.

[] Chris Olah, Nick Cammarata, Ludwig Schubert, Gabriel Goh, Michael Petrov, and Shan Carter. Zoom in: An introduction to circuits. *Distill*, 2020. doi: 10.23915/distill.00024.001. URL https://distill.pub/2020/circuits/zoom-in.

[] Kiho Park, Yo Joong Choe, and Victor Veitch. The linear representation hypothesis and the geometry of large language models, 2023.

[] Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets, 2022.

[] Philip Quirke and Fazl Barez. Understanding addition in transformers. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2310.13121.

[] Philip Quirke, Clement Neo, and Fazl Barez. Understanding addition and subtraction in transformers, 2025.

[] Nick Rimsky, Nick Gabrieli, Julian Schulz, Meg Tong, Evan Hubinger, and Alexander Matt Turner. Steering Llama 2 via contrastive activation addition, 2023.

[] Yucheng Sun, Alessandro Stolfo, and Mrinmaya Sachan. Probing for arithmetic errors in language models, 2025. URL https://arxiv.org/abs/2507.12379.

[] Ziqian Zhong, Ziming Liu, Max Tegmark, and Jacob Andreas. The clock and the pizza: Two stories in mechanistic explanation of neural networks. In *Advances in Neural Information Processing Systems*, 2023. doi: 10.48550/arXiv.2306.17844.

[] Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Hubert Li, Andeng Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, Shashwat Goel, Long Long, Michael J. Byun, Zifan Wang, Alex Mallen, Steven Basart, Sanmi Koyejo, Dawn Song, Matt Fredrikson, J. Zico Kolter, and Dan Hendrycks. Representation engineering: A top-down approach to ai transparency, 2023.

Glossary

Attribution graph A directed acyclic graph whose nodes represent model components (token embeddings, SAE or CLT features, attention heads, output logits) and whose edges represent gradient-weighted information flow between them. Introduced as a methodology by **?**].

Circuit A subgraph of the model's computational graph comprising a small set of components (attention heads, MLP layers, or individual features) that together implement a specific behaviour. The term is used in mechanistic interpretability to denote minimal, interpretable sub-computations.

CLT — Cross-Layer Transcoder A sparse dictionary-learning module, analogous to a sparse autoencoder, that maps MLP inputs at one layer to residual-stream contributions, potentially spanning across layers.

FFN — Feed-Forward Network Synonym for MLP in the transformer literature.

Grokking The phenomenon whereby a neural network trained on a small dataset first memorises the training examples (near-perfect training accuracy, poor generalisation) and then, after many additional gradient steps, abruptly transitions to a generalising solution. Mechanistically characterised for modular arithmetic by **?**].

LLM — Large Language Model A neural language model with a parameter count large enough that emergent capabilities arise from scale alone, typically trained on broad natural-language corpora via next-token prediction.

MLP — Multilayer Perceptron In the transformer architecture, the feed-forward sublayer of each block, consisting of two learned linear projections with a pointwise nonlinear activation in between. Despite the name, transformer MLPs typically have only a single hidden layer.

RMSNorm — Root Mean Square Layer Normalisation A variant of layer normalisation that rescales activations by their root mean square without subtracting the mean. Used in Qwen and other modern architectures in place of standard LayerNorm.

SAE — Sparse Autoencoder A dictionary-learning module trained to decompose neural network activations into a sparse linear combination of learned feature directions. Used in mechanistic interpretability to identify human-interpretable features inside a model.

Lookup table feature A transcoder feature whose activation matrix $M_{\ell,k}[a, b]$ concentrates on the 200 cells satisfying $a \bmod 10 \in \{d_1, d_2\}$ and $b \bmod 10 \in \{d_1, d_2\}$ for a specific ones-digit pair (d_1, d_2). Such features encode the pre-computed result of the ones-digit sum $d_1 + d_2$ and form the mechanistic primitive of addition circuits identified by **?**].

Operand matrix A 100×100 array $M_{\ell,k}[a, b]$ recording the activation of transcoder feature k at layer ℓ at the final input-token position, swept over all integer-pair prompts `calc:` a`+`b`=` with $a, b \in \{0, \ldots, 99\}$. The geometric pattern of this matrix encodes the quantity that the feature tracks.

Representation engineering A methodology for extracting and analysing concept directions in the activation space of a neural network by computing the mean difference in hidden-state vectors between semantically contrasting inputs. Proposed by **?**] as a top-down approach to model transparency, complementary to the bottom-up circuit-based programme.

Sharpness index A scalar summary of how tightly a concept delta's norm is concentrated around its peak layer, defined as the fraction of total norm mass lying within a window of width three centred at the layer of maximum norm. Values close to one indicate a localised representation; values close to $3/L$ indicate a representation that is uniformly distributed across the network's L layers.

Activation patching A causal intervention technique in which a single intermediate representation, here the residual-stream vector at a chosen layer and token position, is replaced by the corresponding value from a different input. The resulting change in output logits quantifies the causal sufficiency of that representation for driving the model's prediction.

Gradient-dot-delta A first-order approximation to activation patching obtained by computing the inner product of the output-margin gradient with respect to a layer's residual stream, evaluated at the baseline input, and the concept delta vector at that layer. The approximation is exact when the output margin is linear in the residual stream; agreement with activation patching in practice confirms that the relevant neighbourhood is approximately linear.

Source-exclusive feature A transcoder feature that is strongly active under a source prompt condition and weakly active under a target condition, qualifying it for suppression during a causal intervention sweep. The complementary notion, a target-exclusive feature, is one that is strongly active in the target condition and weakly active in the source, qualifying it for injection. Formally, feature k at layer ℓ is source-exclusive if $a_{\ell,k}^{\text{tgt}} < \rho \cdot a_{\ell,k}^{\text{src}}$ for a threshold $\rho \in (0, 1)$.

Intervention sweep An experiment in which a causal modification of the model's internal activations is applied at a sequence of increasing strengths γ, and the resulting next-token probabilities for a set of tracked tokens are recorded at each value. A successful sweep produces a monotone crossover between the source-condition and target-condition token probability curves, providing evidence that the modified features causally mediate the targeted aspect of the output.

Operand divergence position The index of the first token at which two prompt sequences differ, used in the operand swap experiment to locate the token position at which the operand concept enters the context. For the English prompts *the opposite of "small" is "* and *the opposite of "hot" is "*, this is the position of the tokens corresponding to "small" and "hot" respectively.